Business Process Management

Campus Enablement Program: Day 1

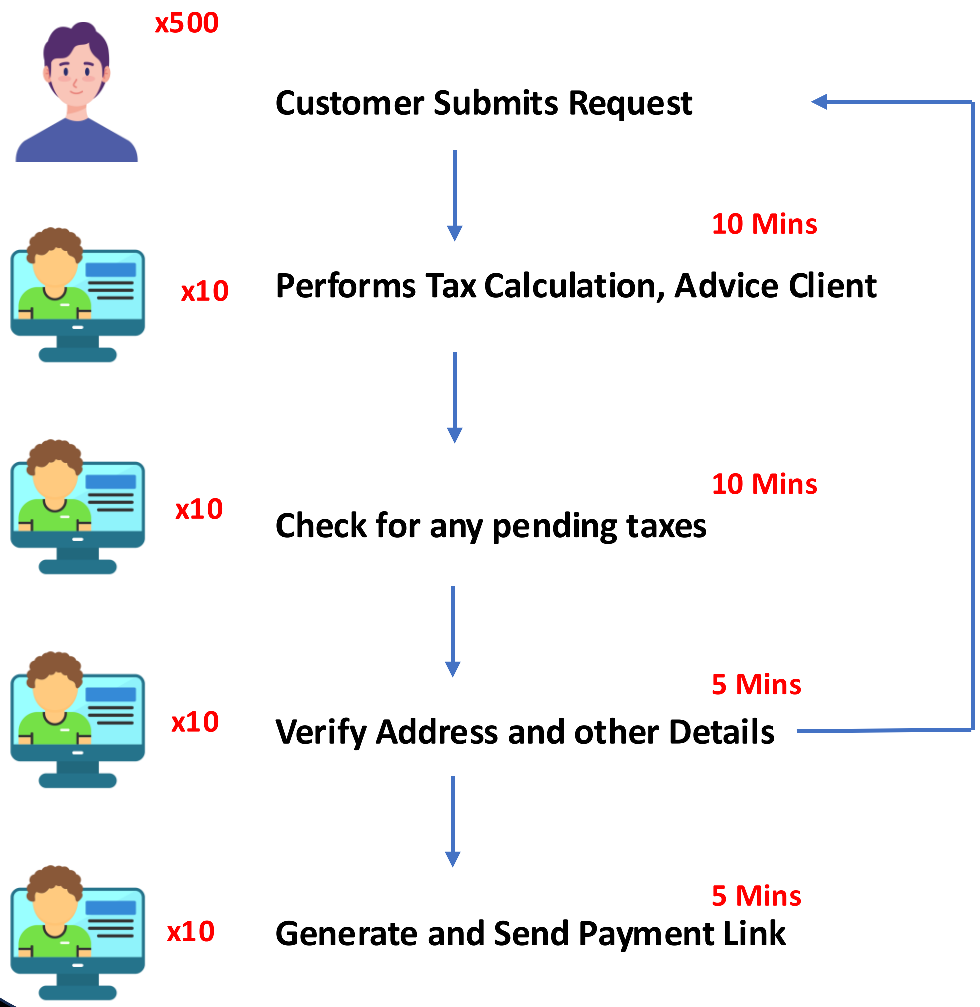
December 2024

MakeLabs.in

Copyright and Documentation Acknowledgment

QUICK EXERCISE

Quick Exercise



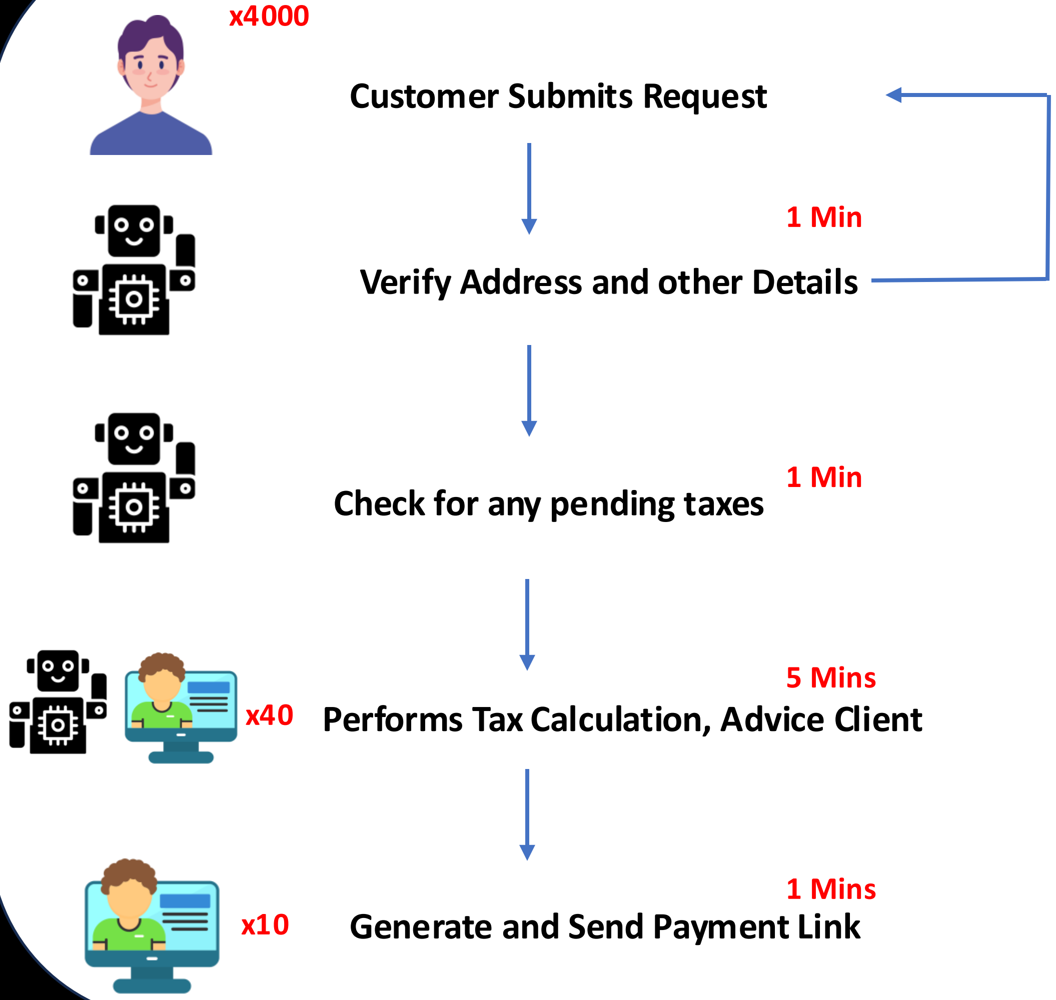
Business pointers and Pain points

- 500 Applications in a Day
- 10 Resources for each step
- Average working hour is 8-10
- Organization is a low-tier with limited budget for digitalization
- Completion rate is 50%
- More than 20,000 applications have been piled up
- Customer retention is reducing 10% YOY

- -> 500 * 30 = 15000 mins = 250 hrs
- -> 40*7 = 280 hrs

Income Range	Tax Rate (%)
$0 - $10,000	0%
$10,001 - $50,000	10%
$50,001 - $100,000	20%
$100,001 and above	30%

Quick Exercise



Important factors to improve inefficiencies

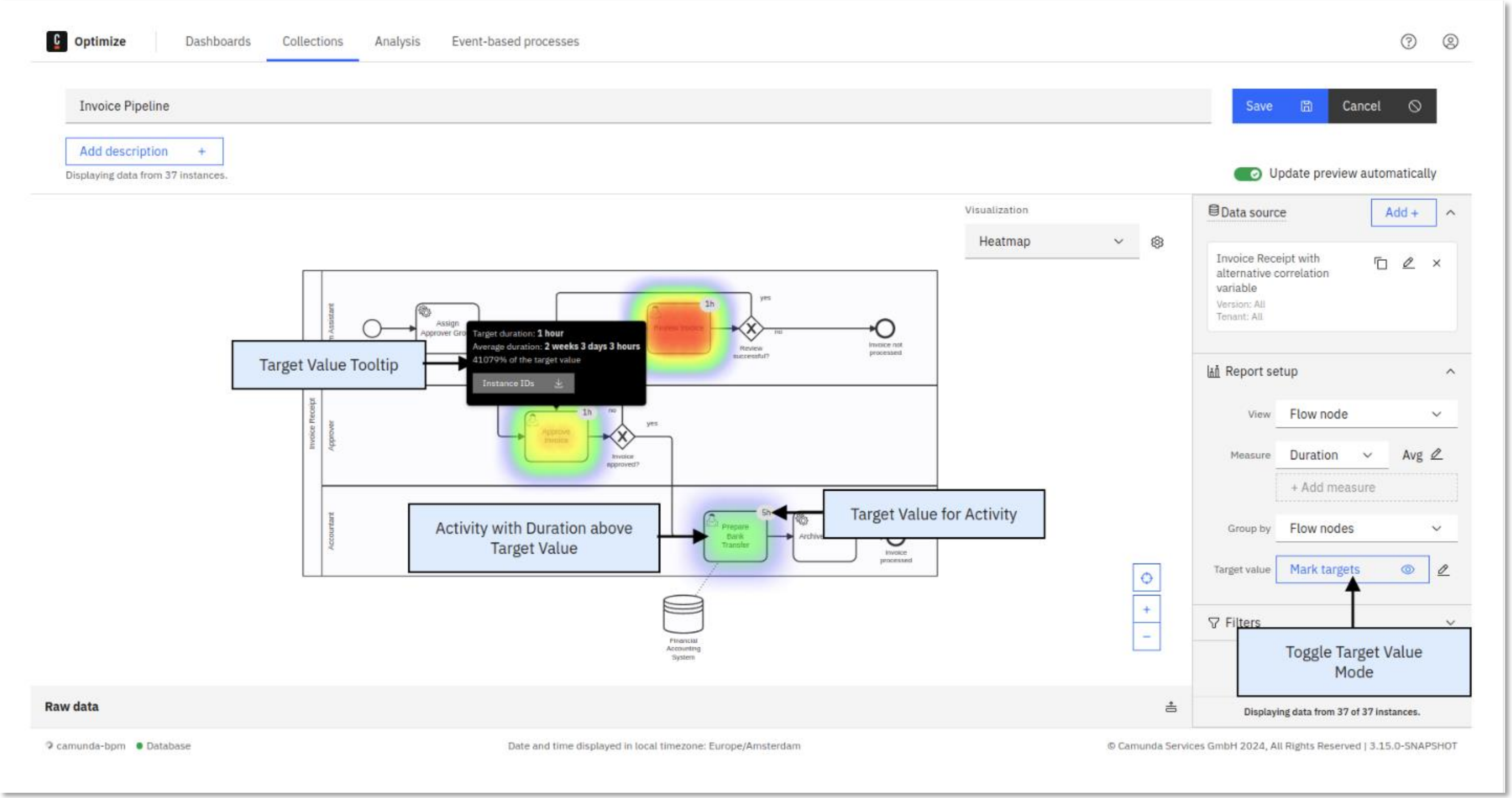
- Analyse bottlenecks and pain points
- Lifting and Shifting of steps to reduce rework
- Resource Optimization
- Finding areas of automation by analysing time per activity

“The first rule of any technology used in a business is that automation applied to an efficient operation will magnify the efficiency.

The second is that automation applied to an inefficient operation will magnify the inefficiency.”

- Bill Gates

BPM Heat MAP



Optimize
Dashboards
Collections
Analysis
Event-based processes
Invoice Pipeline
Save
Cancel
Add description +
Displaying data from 37 instances.
Update preview automatically
Visualization
Heatmap
Data source
Add +
Invoice Receipt with alternative correlation variable
Version: All
Tenant: All
Report setup
View
Flow node
Measure
Duration
Avg
+ Add measure
Group by
Flow nodes
Target value
Mark targets
Filters
Target Value Tooltip
Target duration: 1 hour
Average duration: 2 weeks 3 days 3 hours
41079% of the target value
Instance IDs
Activity with Duration above Target Value
Target Value for Activity
Toggle Target Value Mode
Displaying data from 37 of 37 instances.
Raw data
camunda-bpm
Database
Date and time displayed in local timezone: Europe/Amsterdam
© Camunda Services GmbH 2024, All Rights Reserved | 3.15.0-SNAPSHOT

Case Study

BSI Financial Services drives digital transformation

Challenges

- As a mortgage service company, BSI Financial plays in a complicated ecosystem with many stakeholders, including borrowers, asset sellers
- Regulations that affect financial services. The opportunity for errors is high—as are the ramifications for everyone involved.
- Enormous amounts of data and documentation to support the transfer and define the original terms of the loan.
- Duplicative data, missing data, additional data mapping mismatches, image-to-data extraction errors, dataset merge and service system upload errors.

Solution

- With the key process identified, the team turned to process redesign and orchestration to first automate the most repetitive tasks to reduce errors and increase efficiency
- Find and automate all human tasks in the process that did not explicitly require human intervention or intelligence.

Source: https://camunda.com/case-study/bsi-financial-services/

BSI Financial Services drives digital transformation

KPI

- Increase straight-through processing and reduce manual intervention
- Reduce the time needed to onboard the loan
- Reduce latency, or the amount of time between loan data being delivered and when BSI acts on that data
- Reduce secondary latency, or the time needed to review what has been sent to make sure it's complete
- Reduce errors in the loan onboarding process

Results

- Prior to Camunda, it took an average of 4-5 days to onboard a loan from when the new loan documentation was sent to BSI Financial. With the new orchestrated process leveraging Camunda, onboarding now takes two days or less.
- 60% of overall volume runs straight through, and around 50% of transfers are completed with straight-through processing.
- the orchestrated process is supporting risk reduction efforts by reducing critical errors earlier in the loan lifecycle, stopping problems from compounding further downstream.
- There is approximately a 35% reduction in the cost required to onboard a loan and errors are down to less than 5%

Source: https://camunda.com/case-study/bsi-financial-services/

Business Process Management

Business Process Management

Business Process Management is a systemic approach for designing, modelling, executing, monitoring, and optimizing both automated and non-automated processes to meet the objectives and business strategies of a company.

Embedded business process management not only improves operational efficiency but also **drives growth**, helps manage **governance, risk, and compliance**, and **improves customer experience** and **employee engagement**.

BPM focuses on an entire process rather than individual tasks, which helps strengthen performance, KPIs, and outcomes—and results in fewer errors, happier customers, and lower costs.

Benefits of BPM

- **Model processes.** Using low-code development, multidisciplinary teams that include IT, citizen developers, and business users can use visual interfaces similar to flow diagrams to create process models for their applications. These process models can then be saved for reuse by other applications and faster design in the future.

- **Apply the right worker to the job.** Orchestrate data, workers—both human and digital—and systems across the organization so processes run as efficiently as possible.

- **Apply business rules** to identify, implement, and consistently replicate important policies and procedures (such as spend request thresholds, tax brackets, and approval chains) so you can improve governance and compliance.

- **Adapt to changing situations.** BPM enables organizations to quickly track, analyze, and adapt to evolving conditions, like changes to business events and regulations.

Understanding the Lifecycle

Set Goals

Before starting any BPM initiative, identify the goals and the scope of the work you will begin to. Do not try to boil the ocean; limit process mapping to just enough data to show value quickly. Mapping it can soon become overwhelming, requiring interviewing multiple process experts to gather data about how things work.

Analyze

The goal is to analyze how existing processes work to see if they support the business strategy and work efficiently.

This phase helps understand the root cause of process inefficiencies, highlight bottlenecks, and define how to update the process to improve operational excellence.

Design

Using a process simulation engine, process designers can compare different optimization options and evaluate the impact of changes before implementing them.

By obtaining a visual representation of a process, all stakeholders better understand their role and can more easily collaborate with other employees.

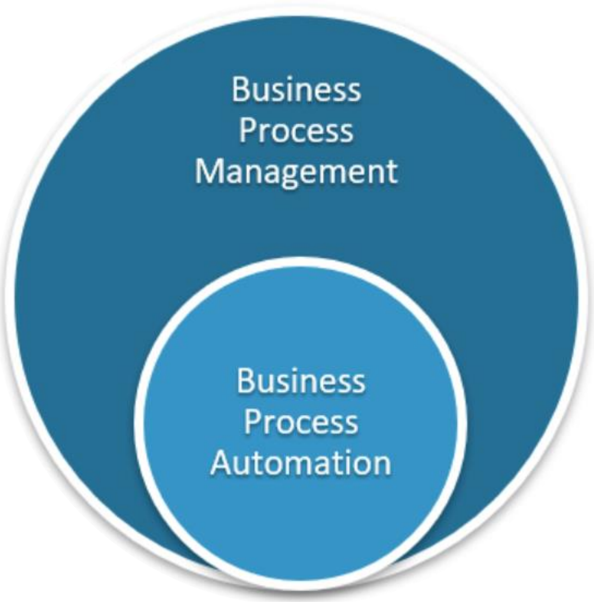
Execute

It is now time to implement the process as has been planned. This phase can only succeed with good change management. Implementing a new strategy often automates part of the process previously done manually, changing how people are used to working. Please address the people's concerns about these changes to ensure the process improvement initiative is successful.

Monitor

Once implemented, it is critical to monitor your new process to see if it is executed as planned, performs as expected, and determines if an additional redesign is required. Improvement is a continuous exercise requiring constant monitoring.

BPM vs BPA



Business Process automation is the automatic orchestration of business processes via a workflow engine. It is a key component of digital transformation. Process automation reduces cost and resource requirements, thereby improving productivity and efficiency and ultimately customer experience.

So, while BPA can generally be described as an organization's efforts to replace manual processes with automation, BPM is an organizational discipline in which a company evaluates and analyzes its business processes to introduce automation that aligns with business strategies.

Can all the processes be automated?

An example of a business process that is standard but not fit for automation is **strategic decision-making in business planning**

Description: This process involves high-level decision-making for setting business goals, determining market strategies, and allocating resources. It includes tasks like brainstorming, analyzing market trends, and conducting discussions among senior executives.

Steps:
1. Gather data from various departments (sales, finance, marketing, etc.).
2. Analyze external factors (market trends, competitor behavior).
3. Conduct brainstorming sessions with stakeholders.
4. Discuss and debate potential strategies.
5. Finalize business goals and strategic plans.

Why It's Not Fit for Automation/Orchestration:

1. **Unstructured and Creative**: Strategic planning requires creativity, innovation, and human intuition that cannot be captured by pre-defined rules or workflows.
2. **High Degree of Variability**: Each instance of this process may involve unique considerations based on external and internal business conditions.
3. **Heavily Dependent on Human Judgment**: Decisions rely on subjective inputs, negotiations, and consensus-building, which are not easily codified into automation logic.
4. **Low Frequency**: Strategic planning happens periodically (e.g., annually or quarterly), and the benefits of automating such an infrequent process are minimal compared to the effort.

Can you think of any process which can't be automated ?

Automation and Orchestration

Design Your First Process

Design Your First Process

Objective

Streamline the onboarding experience for new employees by automating tasks, ensuring compliance, and providing a seamless experience.

Scope

Trigger: Onboarding request is initiated by HR.
Outcome: New employee is fully onboarded, with all systems, accounts, and access set up, and mandatory training completed

Key Steps

HR Request Submission: HR enters the new hire details into the system.
Document Collection: Employee uploads necessary documents (e.g., ID proof, contracts).
Approval: Manager reviews and approves the onboarding request.
System Access Setup: IT creates email accounts, grants system access, and assigns equipment.
Training Assignment: Learning and Development assigns mandatory training.
Compliance Check: Verify background check and document validation.
Welcome Communication: Send onboarding schedule and welcome kit details to the employee.

Requirements

- Capture process timing for SLA reporting.
- Include exception handling for incomplete or rejected approvals.
- Ensure integration with HR and IT systems

Design Your First Process- Extended

SLA Requirements

- **Task Completion Deadlines**:
- Each step in the process must have clearly defined deadlines. For example:
    - Document Collection: 2 business days.
    - Manager Approval: 1 business day.
    - IT System Access Setup: 3 business days.
- **Automated SLA Monitoring**:
- The system must monitor SLAs and notify stakeholders if deadlines are approaching or breached.
- Use color-coded indicators (e.g., green for on track, amber for at risk, red for breached) in the dashboard.
- **Escalation Mechanism**:
- For tasks nearing SLA breaches, send reminders and escalate to the next level (e.g., team lead or manager).
- Example: If an approval task is pending beyond 6 hours of the SLA deadline, escalate to the department head.
- **Reporting and Analytics**:
- Generate real-time SLA compliance reports, including percentage of tasks completed within SLA.
- Provide historical SLA trend analysis to identify bottlenecks.

Design Your First Process- Extended

Exception Requirements

•**Common Exceptions**:
•Missing or incomplete documents during the document collection phase.
•Manager unavailability for approval.
•Errors in data entry (e.g., incorrect employee details).
•IT system failures during access setup.
•**Exception Identification and Notification**:
•The system must detect exceptions in real-time, log them, and notify relevant stakeholders.
•Example: If a document upload fails, notify HR and the new employee immediately.
•**Automated Re-routing**:
•For exceptions such as manager unavailability, automatically re-route tasks to a designated backup approver.
•Allow HR to manually reassign tasks in exceptional cases.
•**Fallback Plans**:
•Define clear fallback actions for each type of exception.
    •Example: If IT cannot set up system access within SLA due to technical issues, provide temporary guest access.
•**Audit Trail**:
•Maintain an audit trail of all exceptions, including timestamps, actions taken, and responsible parties.

Lean and Six Sigma

List of Companies successfully implemented lean or six sigma

Source: https://en.wikipedia.org/wiki/List_of_Six_Sigma_companies
https://boardmix.com/examples/example-of-lean-management/

Lean vs Six Sigma

Lean

•**Origin:** Rooted in **Toyota Production System (TPS)**, developed in post-World War II Japan.

•After World War II, Japan faced a severe economic crisis marked by limited resources, poor industrial infrastructure, and low productivity. Toyota, a small automaker at the time, needed to compete with large Western companies, such as Ford and General Motors, which had abundant resources and economies of scale.

•The system focused on eliminating waste (*muda*), improving quality, and optimizing flow. By implementing Lean, Toyota transformed itself from a small, struggling automaker into a global powerhouse.

•Focuses on the elimination of waste (*muda*) and creating value for the customer by streamlining processes. Lean emphasizes improving flow, reducing delays, and maximizing resource efficiency.

•**The primary goal of Lean is to reduce waste in the process.**

Six Sigma

•**Origin:** Developed at **Motorola** in the 1980s.

•**Key Concepts:** Focuses on reducing variability and defects in processes using data-driven problem-solving and statistical methods. Six Sigma aims to achieve near-perfection, defined as 3.4 defects per million opportunities.

•Six Sigma was introduced in the 1980s at **Motorola**, a time when the company faced declining quality in its products, leading to customer dissatisfaction and financial losses. Motorola needed a systematic, data-driven approach to improve product quality, reduce defects, and restore its competitive position.

•Six Sigma provided Motorola with a powerful framework to improve product quality, reduce costs, and enhance customer satisfaction. It aimed for near-perfection by targeting **3.4 defects per million opportunities (DPMO)** in processes.

•**The primary goal of Six Sigma is to reduce variation in the process.**

Key Contributors - Toyota

1. **Elimination of Waste:**
   1. **Seven Wastes Identified:** Overproduction, waiting, transportation, overprocessing, inventory, motion, and defects.
   2. Toyota minimized these wastes through techniques like Just-In-Time (JIT) production and visual management systems (*kanban*).
2. **Improved Quality:**
   1. Built-in quality control systems (*jidoka*) allowed workers to stop production to address defects, preventing quality issues from spreading.
   2. This proactive approach reduced rework and scrap costs.
3. **Flexibility and Efficiency:**
   1. **Small Batch Production:** Unlike Western manufacturers, Toyota implemented smaller batch sizes, enabling quicker response to customer demand and reducing inventory costs.
   2. **Quick Changeovers:** Methods like *single-minute exchange of dies (SMED)* enabled Toyota to switch production between models quickly, fostering greater product variety.
4. **Employee Involvement:**
   1. Toyota empowered workers to suggest improvements and solve problems through the concept of *kaizen* (continuous improvement).
5. **Cost Reduction and Productivity:**
   1. By reducing waste and optimizing resources, Toyota achieved lower production costs while maintaining high-quality output.
6. **Global Competitive Edge:**
   1. Lean principles allowed Toyota to produce cars faster, cheaper, and with better quality than competitors.

Key Contributors - Motorola

1. **Defect Reduction:**
    1. By identifying and controlling the root causes of defects using tools like **DMAIC** (Define, Measure, Analyze, Improve, Control), Motorola significantly reduced variability in processes.
    2. This led to higher-quality products with fewer errors.
2. **Improved Process Efficiency:**
    1. Six Sigma optimized processes by identifying bottlenecks and inefficiencies.
    2. Streamlining workflows reduced production time and operational costs.
3. **Data-Driven Improvements:**
    1. Statistical tools like **control charts**, **regression analysis**, and **design of experiments (DOE)** enabled Motorola to make informed decisions based on data rather than intuition.
    2. This scientific approach minimized guesswork and ensured consistent results.
4. **Cost Savings:**
    1. Motorola reportedly saved over **$16 billion** in the first ten years of Six Sigma implementation.
5. **Customer Satisfaction:**
    1. Higher-quality products with fewer defects improved customer trust and loyalty.
6. **Cultural Transformation:**
    1. Employees were trained in Six Sigma methodologies, creating a workforce capable of driving innovation and process excellence.
7. **Industry Leadership:**
    1. Motorola became a benchmark for quality, winning the **Malcolm Baldrige National Quality Award** in 1988.
    2. This recognition further solidified the company's reputation as a quality leader.

Lean Six Sigma

A variation on the Six Sigma framework marries Lean Management principles with Six Sigma methodologies.

Six Sigma focuses on finding and fixing defects after the fact, while Lean tries to prevent defects from occurring in the first place. At its best implementation, Lean is introduced first to increase efficiency, and then Six Sigma methods are applied for fine-tuning. Manufacturing, healthcare, finance, IT, and other fields use Lean Six Sigma.

Understanding DMAIC with Example

Problem Statement

A university cafeteria receives complaints from students about long waiting times during lunch hours. Despite having enough staff, many students report delays in receiving their meals, leading to dissatisfaction and overcrowding. The university administration wants to address this issue while ensuring quality service and minimizing food waste.

Objective

Reduce the average waiting time for students during peak lunch hours from **15 minutes** to **5 minutes** while maintaining service quality and reducing operational inefficiencies.

Understanding DMAIC with Example - Define

Purpose: Clearly identify the problem, understand customer needs, and define the project scope and goals.

Key Deliverables:

- **Project Charter**: A document that outlines the problem, objectives, scope, team roles, and timeline.
- **Voice of the Customer (VoC)**: Gather customer feedback to understand needs and pain points.
- **SIPOC Diagram**: A high-level process map to identify Suppliers, Inputs, Process, Outputs, and Customers.

Tools Used:

- **Project Charter**: Includes Problem Statement, Goal Statement, Scope, Timeline, and Team.
- **VoC Analysis**: Surveys, interviews, or focus groups to capture customer expectations.
- **SIPOC Diagram**: Visualizes the process flow and relationships.

Outputs:

- Defined problem and goals.
- High-level process understanding.
- Customer-focused insights.

Understanding DMAIC with Example – Define (Project Charter)

Project Title:
Reducing Long Waiting Times in the University Cafeteria

Problem Statement:
Students and staff frequently experience long waiting times in the university cafeteria, particularly during peak hours (12:00 PM–2:00 PM). This issue leads to customer dissatisfaction, reduced productivity, and overcrowding. Current waiting times average **20 minutes**, significantly higher than the targeted **10 minutes**.

Goal Statement:
Reduce average waiting times in the university cafeteria from **20 minutes to 10 minutes or less** during peak hours within **12 weeks**.

Scope:
In-Scope:
•Cafeteria processes for ordering, preparation, and payment during peak lunch hours.
•Interactions between staff and customers during the order cycle.
Out-of-Scope:
•Other dining facilities on campus.
•Staff scheduling outside of peak hours.

Business Case:
Reducing waiting times will enhance customer satisfaction, improve operational efficiency, and align with the university's goal of delivering a better student and staff experience. This project has the potential to:
•Reduce complaints by **50%**.
•Improve cafeteria throughput by serving **30% more customers** during peak hours.
•Free up valuable seating space to accommodate more customers.

Understanding DMAIC with Example – Define (Project Charter)

Key Metrics (KPIs):

- **Baseline Waiting Time**: 20 minutes
- **Target Waiting Time**: ≤ 10 minutes
- **Customer Satisfaction Score**: Improve from **3.5/5** to **4.5/5**
- **Process Cycle Efficiency**: Increase from **40% to 70%**

Team Members and Roles:

Role	Name	Responsibility
Project Sponsor	Cafeteria Manager	Provides resources and oversight.
Project Lead	Lean Six Sigma Student	Leads DMAIC methodology execution.
Data Analyst	Business Analytics Student	Collects and analyzes process data.
Cafeteria Staff	Shift Supervisors	Provides process insights and feedback.
Customers	Students & Staff	Provide VoC through surveys and feedback.

Milestones and Timeline:

Phase	Key Activities	Timeline
Define	Develop charter, VoC analysis, SIPOC	Week 1–2
Measure	Data collection, baseline analysis	Week 3–4
Analyze	Root cause identification (e.g., Fishbone)	Week 5–6
Improve	Pilot solutions, implement improvements	Week 7–10
Control	Standardize, monitor performance	Week 11–12

Risks and Mitigation:

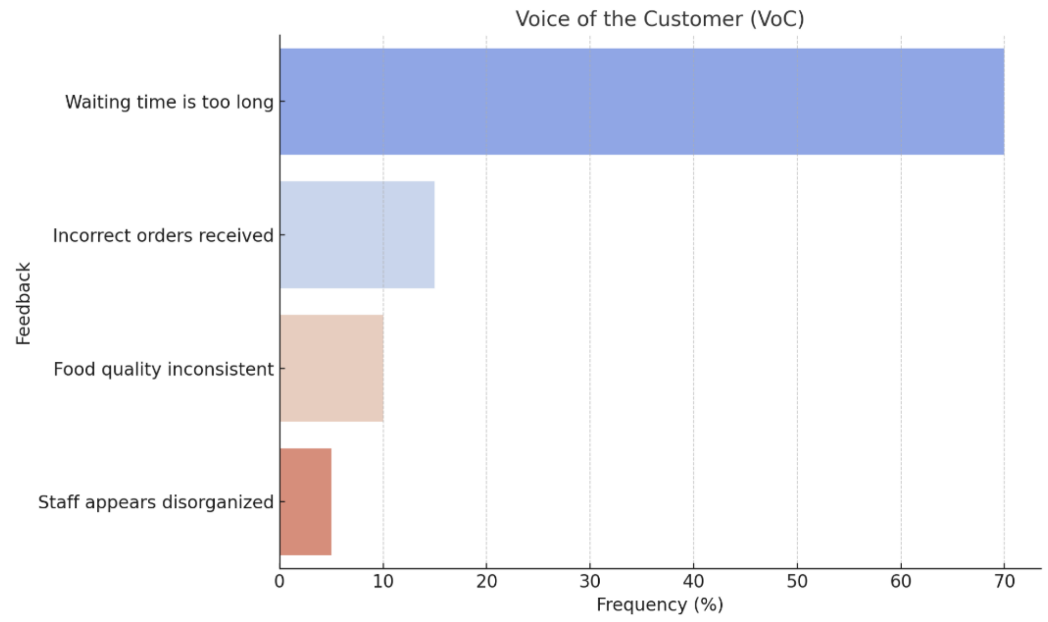
Risk	Mitigation Plan
Resistance from cafeteria staff	Conduct training sessions and involve staff early.
Inaccurate data collection	Validate measurements with multiple observations.
Limited customer participation in surveys	Use incentives like discounts to encourage feedback.

Sign-off:

Stakeholder	Signature	Date
Cafeteria Manager		
Lean Six Sigma Coach		
University Admin Rep		

Understanding DMAIC with Example – Define (VOC & CTQ)

Voice of the Customer (VoC): Captures feedback from students about cafeteria delays.



VoC Table

Feedback	Frequency	Key Concern
Waiting time is too long	70%	Speed of service
Incorrect orders received	15%	Order accuracy
Food quality inconsistent	10%	Food quality
Staff appears disorganized	5%	Staff training

Critical to Quality (CTQ):

Key metrics derived from VoC.

CTQ Metrics

- Waiting time < 5 minutes.
- Order accuracy > 99%.
- Customer satisfaction > 90%.

Understanding DMAIC with Example – Define (Cycle Time)

Cycle time refers to the total time taken to complete one full process cycle, from the beginning of a task to its completion, for a single unit of work. It is a key metric in process improvement methodologies like Lean and Six Sigma, as it helps to measure and optimize the efficiency of a process.

For example, in a cafeteria, **cycle time** could be the time it takes from when a customer enters the queue until they receive their order.

Cycle Time Formula:

$$\text{Cycle Time} = \frac{\text{Available Time}}{\text{Number of Units Produced}}$$

- **Available Time**: The total time available for production or service (e.g., 8 hours of cafeteria operation).
- **Number of Units Produced**: The number of items processed in that time (e.g., number of meals served).

- **Available Time**: The total time available for production or service (e.g., 8 hours of cafeteria operation).
- **Number of Units Produced**: The number of items processed in that time (e.g., number of meals served).

- **Cycle Time ≠ Lead Time**:
  - **Cycle Time**: Focuses only on the active work being done to complete a single unit.
  - **Lead Time**: Includes both cycle time and waiting/delay times

Understanding DMAIC with Example – Define (Cycle Efficiency)

Cycle Efficiency (also called **Process Cycle Efficiency**, or **PCE**) is a measure of how effectively a process converts inputs into valuable outputs by focusing on the proportion of time spent on **value-added activities** compared to the total cycle time. It is a key metric in Lean Six Sigma to identify waste and improve efficiency.

For example, in a cafeteria, **cycle time** could be the time it takes from when a customer enters the queue until they receive their order.

Formula:

$$\text{Cycle Efficiency (\%)} = \left(\frac{\text{Value-Added Time}}{\text{Total Cycle Time}}\right) \times 100$$

•**Value-Added Time**: The time spent on activities that directly contribute to delivering what the customer wants (e.g., preparing food in the cafeteria).
•**Total Cycle Time**: The sum of value-added and non-value-added times, including delays, waiting, or rework.

•**Value-Added Activities:** Actions that directly enhance the product or service for the customer. For example:

•Cooking a meal.
•Packing an order.

•**Non-Value-Added Activities:** Actions that do not add value but are part of the current process. Examples include:

•Waiting in line.
•Unnecessary movement of staff.
•Double-checking orders due to errors.

Understanding DMAIC with Example – Define (SIPOC)

Supplier	Input	Process	Output	Customer
Students	Food orders, preferences	1. Students place orders at the counter.	Orders placed	Students
Staff	Order details, payment method	2. Staff receives and inputs order into system.	Processed orders	Students
Kitchen Team	Order, ingredients	3. Kitchen team prepares food according to order.	Prepared food	Staff, Students
Equipment/Technology	Order system, kitchen equipment	4. Food is delivered to counter for pickup.	Completed orders ready for delivery	Students, Staff
Suppliers (Food Providers)	Ingredients, packaging	5. Order is handed to students.	Served food	Students, Staff

Understanding DMAIC with Example - Measure

Purpose: Establish baseline performance and validate measurement systems to quantify the problem.

Key Deliverables:
•Baseline Metrics: Current performance (e.g., defects, cycle time, etc.).
•Data Collection Plan: Specify what data to collect, how, and from where.
•Process Map: Detailed view of the process to identify inefficiencies.

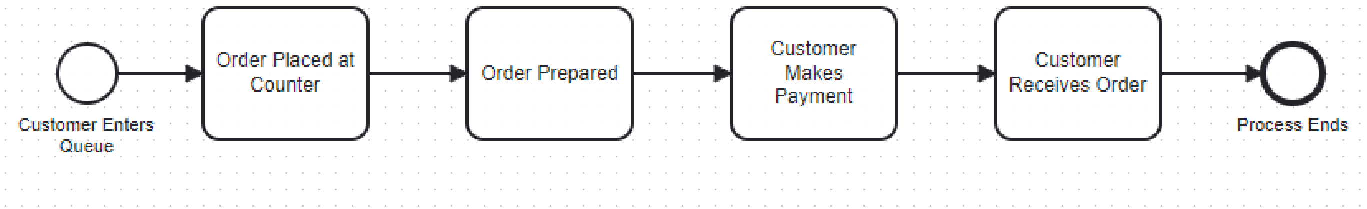
Subcomponents:

1. **Measurement System Analysis (MSA)**:
    1. Ensures measurement systems are accurate and consistent.
    2. Includes tools like Gage R&R (Repeatability & Reproducibility).
2. **Data Collection**:
    1. Collect historical or real-time data relevant to the problem.
    2. Identify critical inputs (X) and outputs (Y).

Tools Used:
•**Pareto Chart**: Prioritize problems based on their frequency or impact.
•**Process Capability Analysis**: Measure how well the process meets specifications.
•**Baseline Metrics**: Current defect rates, cycle times, etc.

Outputs:
•Verified and validated data.
•Process performance baselines.
•Identification of key performance gaps.

Understanding DMAIC with Example – Measure (Process Design)



1. Customer enters the queue

•**Input**: Customer arrives.
•**Output**: Added to the queue.
•**Measure**: Time spent waiting in line.

2. Order is placed at the counter

•**Input**: Customer communicates their order.
•**Output**: Order ticket generated.
•**Measure**: Time taken for order placement.

3. Order is prepared

•**Input**: Order ticket reaches kitchen staff.
•**Output**: Completed food item.
•**Measure**: Time spent on food preparation.

4. Customer makes payment

•**Input**: Order is ready, and payment is requested.
•**Output**: Payment completed.
•**Measure**: Time taken to process the payment.

5. Customer receives the order and exits

•**Input**: Payment is verified.
•**Output**: Customer leaves with the order.
•**Measure**: Total time from entering the queue to receiving the order.

Understanding DMAIC with Example – Measure

Baseline Metrics

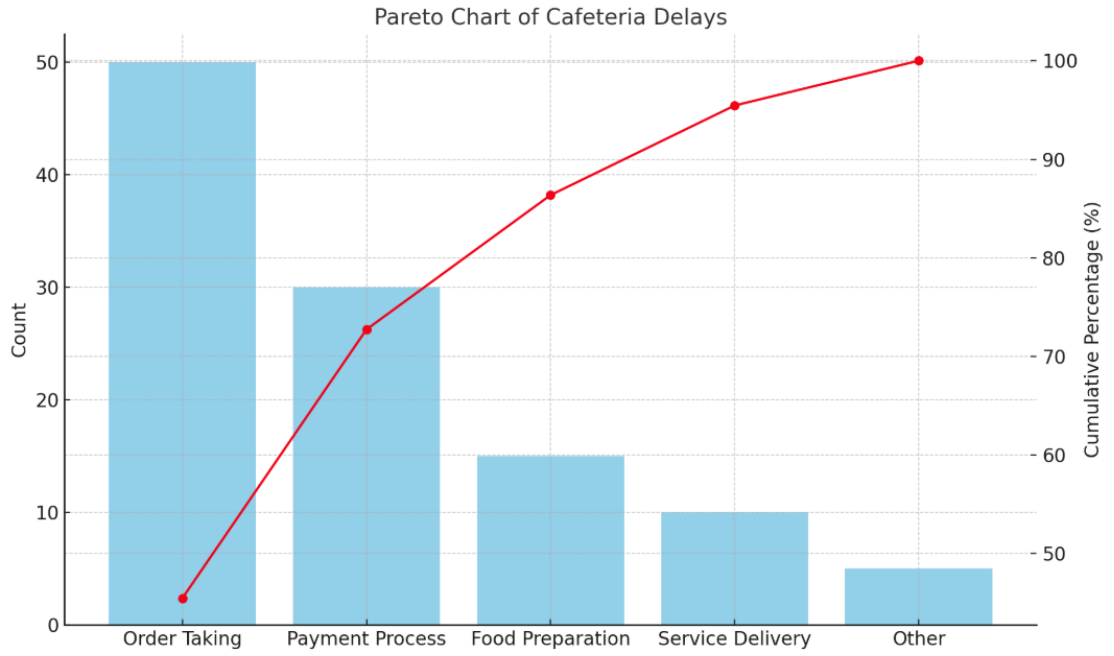
Metric	Baseline Value	Data Source
Average Waiting Time	20 minutes	Observations at peak hours (12 PM–2 PM).
Process Cycle Efficiency	40%	Calculated (value-added time ÷ total time).
Order Accuracy	90%	Customer feedback or error logs.
Customer Satisfaction Score	3.5/5	Survey responses.

Data Collection Framework

Parameter	Details
What to Measure	Waiting times, order accuracy, customer satisfaction.
Where to Measure	At the university cafeteria during peak hours.
When to Measure	Over 5 consecutive weekdays (12 PM–2 PM).
Who Collects the Data	Team members assigned as observers (students or staff).
How to Measure	Stopwatch for timing, surveys for satisfaction, logs for errors.

Understanding DMAIC with Example – Measure (Pareto Chart)

The red line helps pinpoint the **80/20 rule** (Pareto Principle), showing which categories (typically the first few) contribute to approximately **80% of the overall effect**.



Calculation Guide:

1. Start with the percentage contribution of each category.
2. Add the percentage of each subsequent category to create a running total (cumulative percentage).
3. Plot this cumulative percentage as the red line on the secondary axis (Y-axis, often on the right).

Reason	Frequency	Percentage	Cumulative %
Long preparation	50	50%	50%
Queueing	30	30%	80%
Payment delays	15	15%	95%
Others	5	5%	100%

Understanding DMAIC with Example - Analyse

Purpose: Identify root causes of the problem using data analysis and root cause techniques.

Key Deliverables:
•Root Cause Hypothesis: Possible reasons for the problem.
•Cause-and-Effect Analysis: Understanding relationships between factors and the problem.

Subcomponents:

1. Fishbone Diagram (Ishikawa):
   1. Organizes potential causes into categories like People, Process, Technology, etc.

2. 5 Whys Analysis:
   1. Iterative questioning to drill down into the root cause.

Tools Used:

•**Fishbone Diagram**: Visualize root causes.
•**Regression Analysis**: Identify relationships between variables.
•**Hypothesis Testing**: Validate which factors impact the problem.

Outputs:

•Validated root causes.
•Clear understanding of cause-and-effect relationships.

Understanding DMAIC with Example – 5 Why's

1. Why are customers experiencing long waiting times?
There is a delay in preparing orders.

2. Why is there a delay in preparing orders?
The kitchen staff is overwhelmed during peak hours.

3. Why is the kitchen staff overwhelmed during peak hours?
There is no optimized scheduling for staff based on customer volume.

4. Why is there no optimized scheduling for staff?
The cafeteria does not analyze peak and off-peak times to align staffing levels.

5. Why does the cafeteria not analyze peak and off-peak times?
There is no data collection or system in place to monitor customer flow and order patterns.

Root Cause Identified:
Lack of data-driven scheduling and order pattern analysis leads to an overwhelmed kitchen staff during peak hours.

Proposed Solution:
1. Implement a data collection system to monitor customer flow and peak hours.
2. Use historical data to create staff schedules that align with demand.
3. Train additional staff to handle peak-hour workloads.

Understanding DMAIC with Example – 5 Why's

1. Why are customers experiencing long waiting times?
There is a bottleneck at the order-taking counter.

2. Why is there a bottleneck at the order-taking counter?
The cashier spends too much time entering orders manually into the system.

3. Why does the cashier spend too much time entering orders manually?
The ordering system is outdated and not user-friendly.

4. Why is the ordering system outdated and not user-friendly?
The cafeteria management has not upgraded the system in the past five years.

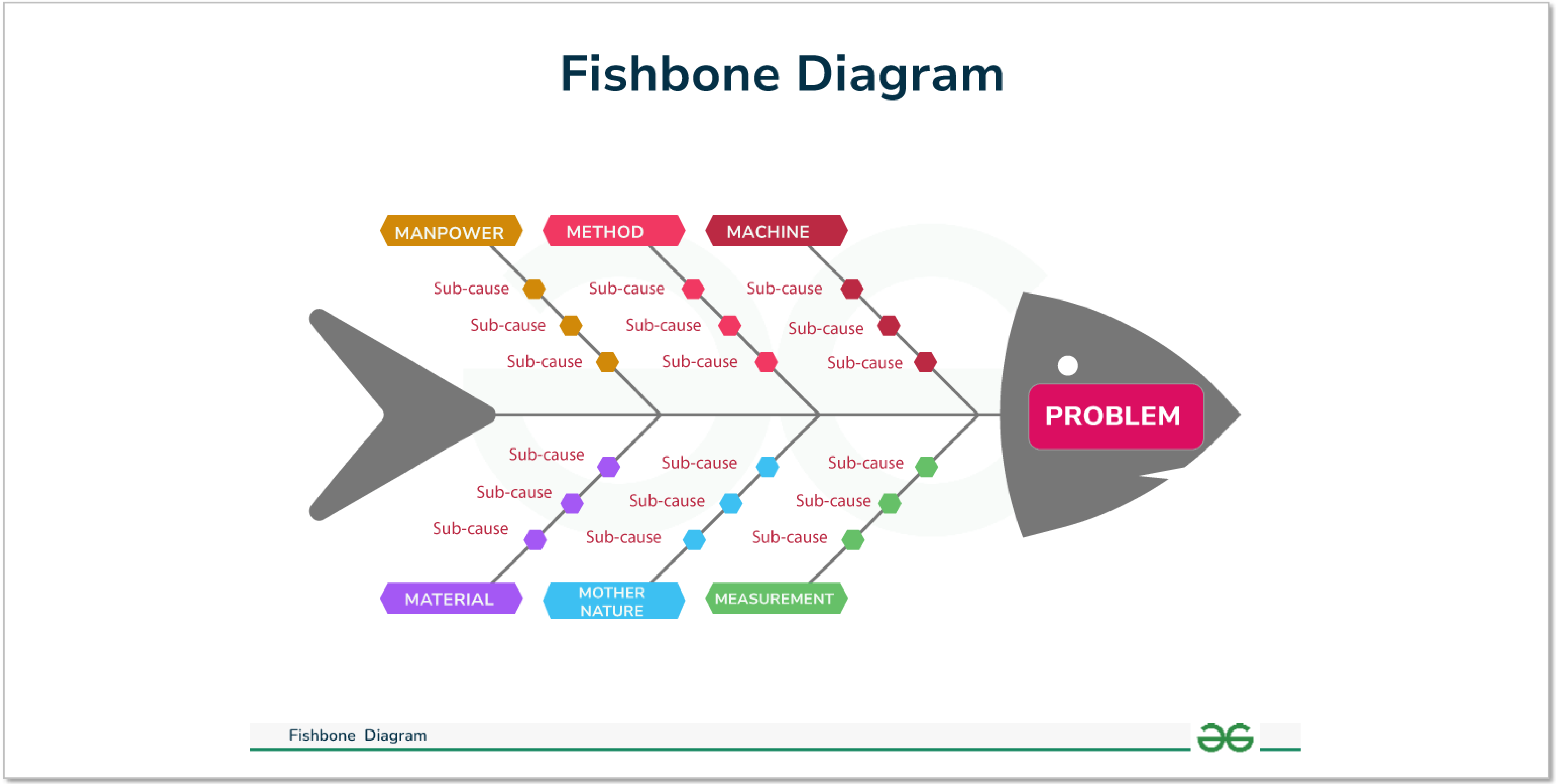
5. Why has the system not been upgraded in the past five years?
There is no budget allocated for system improvements or technological upgrades.

Root Cause Identified:
Lack of investment in upgrading the order-taking system, leading to inefficiencies and delays at the counter.

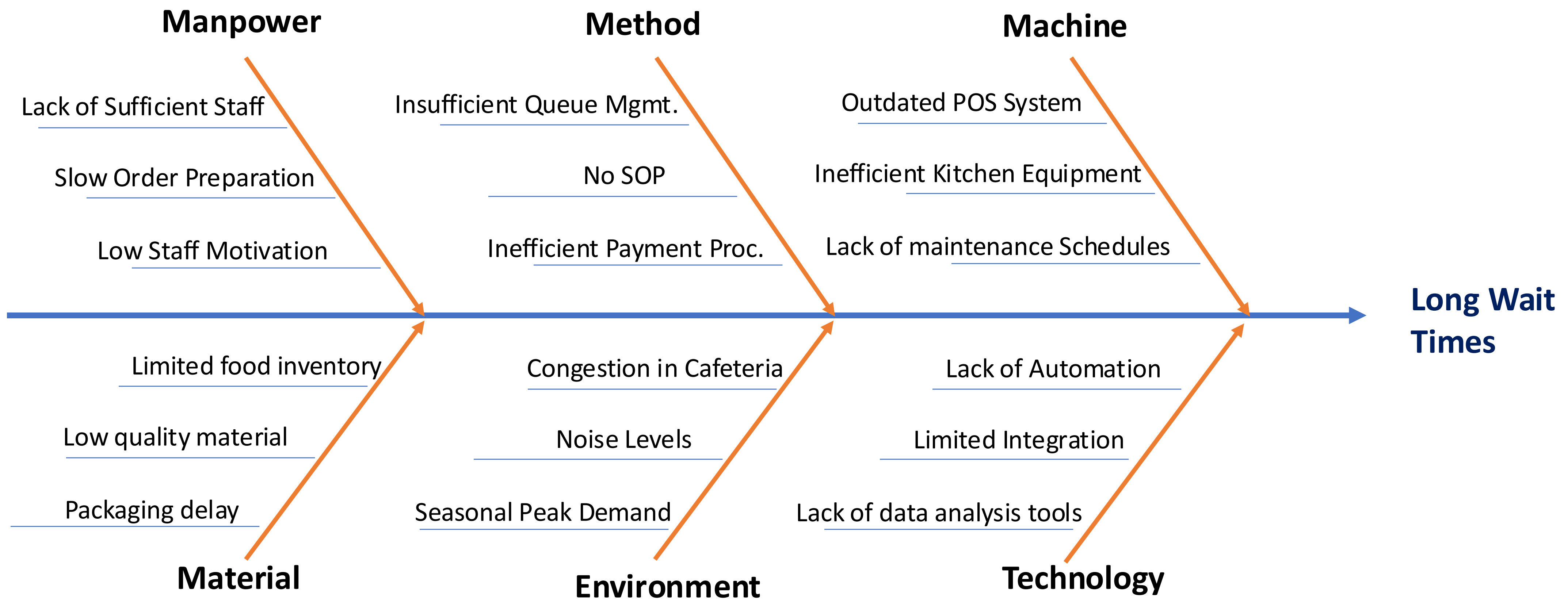
Proposed Solution:

1. Upgrade to a faster, user-friendly Point of Sale (POS) system.
2. Implement self-service kiosks to take orders during peak hours.
3. Train staff to reduce manual errors and speed up the order-taking process.

Understanding DMAIC with Example – Fishbone(Ishikawa)



Understanding DMAIC with Example – Fishbone(Ishikawa)



Understanding DMAIC with Example – Fishbone(Ishikawa) Details

Category	Root Cause	Sub-Causes
Manpower	Lack of sufficient staff	- Inadequate staffing during peak hours.
		- High employee turnover leading to staffing shortages.
	Slow order preparation	- Lack of familiarity with menu items or processes.
		- Staff fatigue due to extended working hours.
	Low staff motivation	- Absence of incentives or rewards for good performance.
		- Poor communication between staff and management.
Method	Inefficient queue management	- No clear signage or instructions for customers.
		- Customers don't know where to stand or how to place orders.
	Delays in order preparation	- Complex menu items slowing down the preparation process.
		- No standard operating procedure (SOP) for high-demand periods.
	Inefficient payment process	- Manual processing of orders and payments instead of automated systems.
Machine	Outdated POS system	- Slow response times during order input.
		- Limited capacity to handle simultaneous transactions.
	Inefficient kitchen equipment	- Equipment breakdowns delaying food preparation.
		- Old appliances consuming more time to prepare food.
	Lack of maintenance schedules	- Kitchen appliances not serviced regularly, leading to unexpected failures.

Understanding DMAIC with Example – Fishbone(Ishikawa) Details

Category	Root Cause	Sub-Causes
Material	Limited food inventory	- Running out of key ingredients during peak hours.
		- Poor inventory management practices.
	Low quality of materials	- Substandard raw materials causing delays in preparation.
	Packaging delays	- Packaging materials not readily available when orders are ready.
Environment	Congestion in cafeteria	- Overcrowded seating areas due to inadequate space.
		- Poor ventilation and lighting increasing stress for staff and customers.
	Noise levels	- Miscommunication between staff due to loud ambient noise.
	Seasonal peak demand	- No specific plan for handling high customer volumes during festivals or university events.
Technology	Lack of automation	- No self-ordering kiosks or mobile app for pre-orders.
		- No digital queue management system to reduce customer anxiety.
	Limited integration	- POS system not integrated with kitchen display systems (KDS).
	Lack of data analysis tools	- No tools for monitoring peak times or analyzing customer flow patterns.

Understanding DMAIC with Example - Improve

Purpose: Develop and implement solutions to eliminate root causes and improve the process.

Key Deliverables:
•Action Plan: Detailed steps to implement solutions.
•Pilots or Prototypes: Testing solutions on a small scale before full implementation.

Subcomponents:

1. **Brainstorming**:
   1. Generate creative solutions to address root causes.
2. **FMEA (Failure Modes and Effects Analysis)**:
   1. Anticipate potential failures in the solutions and address risks.

Tools Used:

•**Design of Experiments (DoE)**: Test multiple factors to optimize the process.
•**Poka-Yoke (Error Proofing)**: Create systems to prevent errors.
•**Kaizen Events**: Rapid, focused improvement sessions.

Outputs:
•Implemented solutions.
•Improved process performance metrics.
•Reduced defects or delays.

Understanding DMAIC with Example – Improve - FMEA

FMEA (Failure Modes and Effects Analysis)

Process Step	Potential Failure Mode	Effect of Failure	Severity (S)	Occurrence (O)	Detection (D)	Risk Priority Number (RPN)	Recommended Action
Wait in Line	Long queues	Customer dissatisfaction	8	7	5	280	Add queue management systems
Place Order	Incorrect order entry	Wrong order served	9	6	6	324	Upgrade POS system
Prepare Order	Delay in preparation	Increased waiting time	8	8	5	320	Optimize kitchen layout
Make Payment	Slow payment processing	Frustration at checkout	7	5	4	140	Introduce contactless payment

Understanding DMAIC with Example – Improve - FMEA

Severity (S)

Severity assesses the **impact** of the failure on the process or customer if it occurs.
•**Scale**: Typically rated from **1 to 10**, where:
- **1** = Negligible impact (little to no effect on the process).
- **10** = Catastrophic impact (severe consequences, e.g., major safety issues or customer dissatisfaction).

•**Example for Cafeteria**:
- Incorrect order (customer gets wrong meal): **S = 6** (moderate customer inconvenience).
- Equipment failure causing delays: **S = 8** (high customer dissatisfaction).

Occurrence (O)

Occurrence evaluates how **likely** the failure is to happen.
•**Scale**: Typically rated from **1 to 10**, where:
- **1** = Highly unlikely (failure is rare).
- **10** = Highly likely (failure is almost certain).

•**Example for Cafeteria**:
- Long queue during peak hours: **O = 7** (common issue).
- System crash: **O = 4** (less frequent but possible).

Detection (D)
Detection measures the **likelihood of identifying the failure** before it affects the customer.
•**Scale**: Typically rated from **1 to 10**, where:
- **1** = Very likely to detect (failure is easily caught).
- **10** = Very unlikely to detect (failure is rarely or never caught).

•**Example for Cafeteria**:
- Order entry errors: **D = 6** (manual input may go unnoticed).
- Equipment failure: **D = 3** (technicians often detect issues during regular maintenance).

Risk Priority Number (RPN)

RPN is the product of **S, O, and D**:
•RPN=S×O×D\text{RPN} = S \times O \times DRPN=S×O×D
•**Purpose**: RPN helps prioritize which failure modes to address first. Higher RPN values indicate greater risk.
•**Scale**: RPN ranges from **1 to 1000**.
•**Example Calculation**:
- **Severity (S) = 8**, **Occurrence (O) = 7**, **Detection (D) = 5 -> 280**

Understanding DMAIC with Example – (Kaizen, DOE and Poka-Yoke)

Design of Experiments (DOE)

Experiment Objective:

Determine the impact of staff allocation and order-processing technology on reducing waiting times.

Factors & Levels:

Factor	Low Level	High Level
Staff allocation	2 staff members	4 staff members
POS system upgrade	Manual	Automated

Experiment Design: Full Factorial Design

•4 Runs:
- 2 staff + Manual system
- 2 staff + Automated system
- 4 staff + Manual system
- 4 staff + Automated system

Result: The combination of **4 staff** and an **automated system** resulted in the highest reduction in waiting times.

Kaizen

Small Continuous Improvements:

•Rearrange kitchen stations to reduce movement and improve food preparation speed.
•Use simple labels and color-coding for order prioritization (e.g., peak hour orders marked with a red tag).
•Set up a feedback mechanism for real-time issue reporting by customers.

Poka-Yoke (Error Proofing)

Small Continuous Improvements:

•**Order Accuracy:** Design POS systems to prompt staff for double confirmation before finalizing the order.
•**Queue Management:** Add visual and audio alerts when queue length exceeds a set threshold.
•**Payment:** Use contactless payment systems to eliminate manual errors in cash handling.

Understanding DMAIC with Example - Control

Purpose: Sustain the improvements by standardizing the process and monitoring performance.

Key Deliverables:
- Control Plan: Documented methods to sustain changes.
- Monitoring System: Tools to track performance and detect deviations.

Subcomponents:

1. **Statistical Process Control (SPC)**:
    1. Use control charts to monitor stability.
2. **Standard Operating Procedures (SOPs)**:
    1. Document the new process to ensure consistency.

Tools Used:

- **Control Charts**: Monitor ongoing performance.
- **Checklists**: Standardize tasks and ensure compliance.
- **Audits**: Periodic checks to verify adherence to the improved process.

Outputs:

- Stabilized and improved process.
- Handover to process owners.
- Monitoring framework to ensure sustained performance.

Understanding DMAIC with Example - Control

Key Metrics to Monitor

Metric	Baseline Value	Target Value	Frequency of Measurement
Average waiting time	20 minutes	<5 minutes	Daily
Order accuracy	90%	>99%	Weekly
Customer satisfaction score	70%	>90%	Monthly
Staff utilization rate	60%	85%	Weekly

Control Chart

- Sample Data (Post-Implementation):

```yaml
Week 1: 4.8, 5.0, 4.5, 4.9, 5.2, 4.7, 4.6
Week 2: 4.3, 4.8, 4.6, 4.9, 4.5, 4.7, 4.4
```

- **Steps:**

1. Calculate the **mean (central line)** of the data.

$$\text{Mean} = \frac{\text{Sum of all values}}{\text{Number of values}}$$

   Mean = 4.7 minutes.

2. Calculate the **Upper Control Limit (UCL)** and **Lower Control Limit (LCL)**:

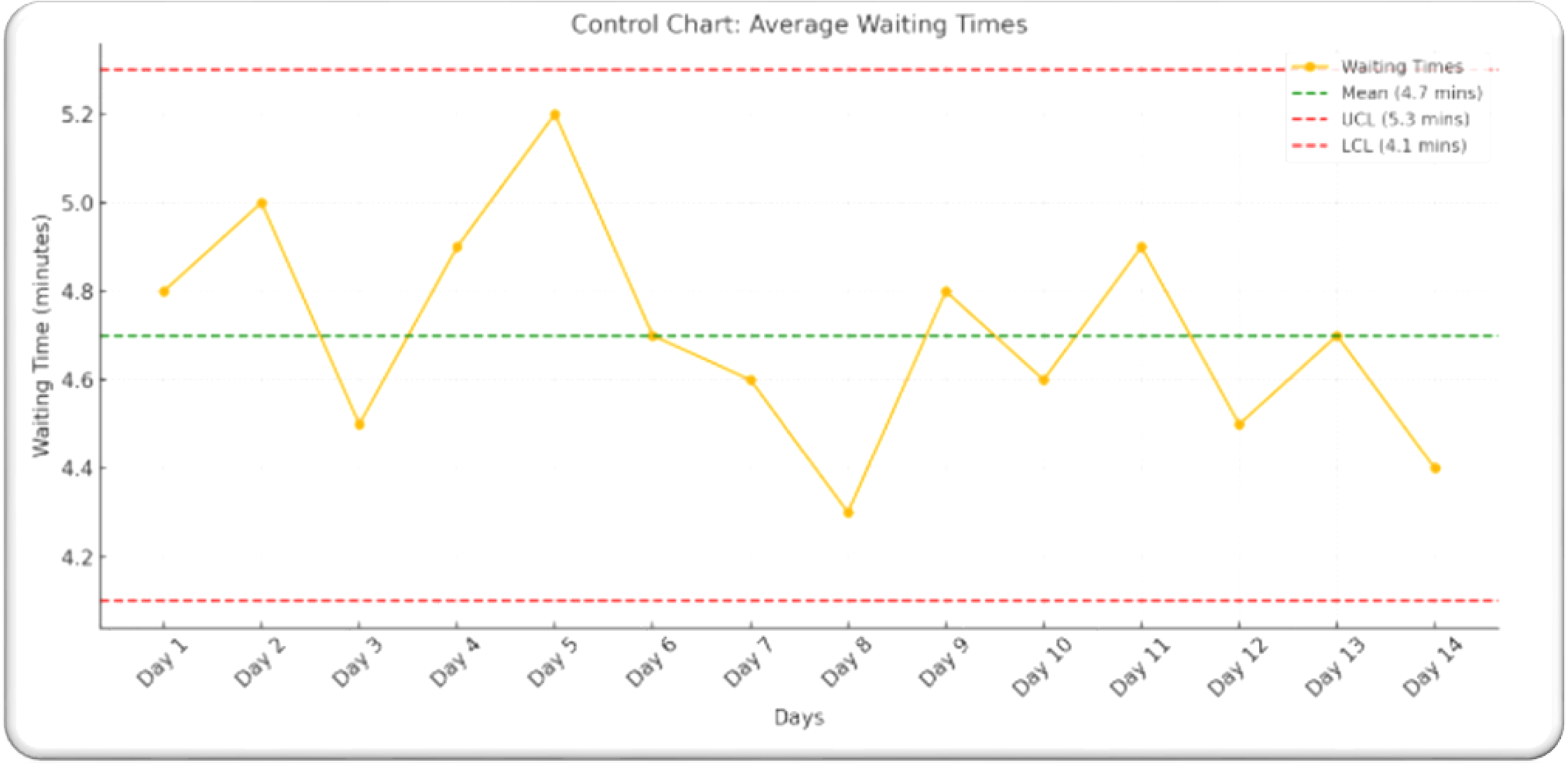
$$UCL = \text{Mean} + (3 \times \sigma), \quad LCL = \text{Mean} - (3 \times \sigma)$$

   - σ (standard deviation) = 0.2.
   - $UCL = 4.7 + (3 \times 0.2) = 5.3.$
   - $LCL = 4.7 - (3 \times 0.2) = 4.1.$

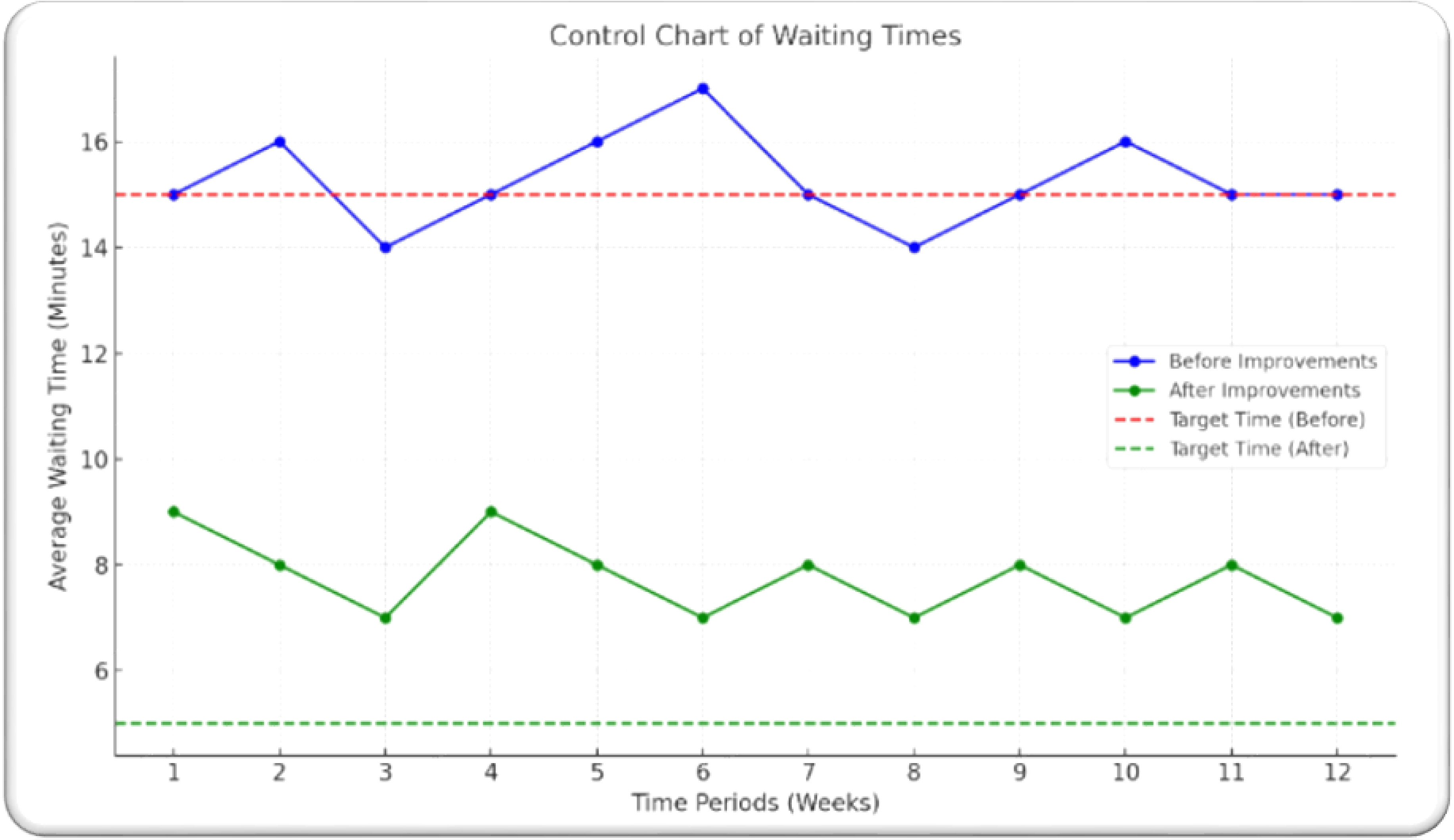
Interpretation:

- If waiting times fall within the control limits (4.1 to 5.3), the process is stable.
- Data points outside these limits indicate special causes requiring corrective actions.

Understanding DMAIC with Example - Control



Understanding DMAIC with Example - Control

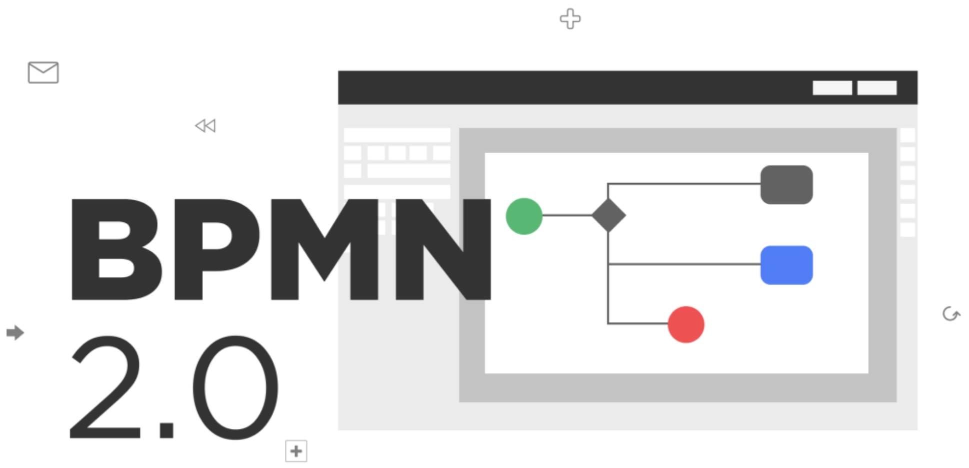


Summary Table

Phase	Purpose	Key Tools	Output
Define	Identify the problem & scope	Project Charter, VoC, SIPOC	Defined problem, customer insights
Measure	Establish baseline performance	Pareto Chart, Process Map, MSA	Validated data, baseline metrics
Analyze	Find root causes	Fishbone Diagram, 5 Whys, Hypothesis Test	Validated root causes
Improve	Implement solutions	FMEA, DoE, Poka-Yoke	Improved process metrics, action plan
Control	Sustain improvements	SPC, SOPs, Control Charts	Stabilized process, monitoring framework

BPMN 2.0 Guide

What is BPMN?



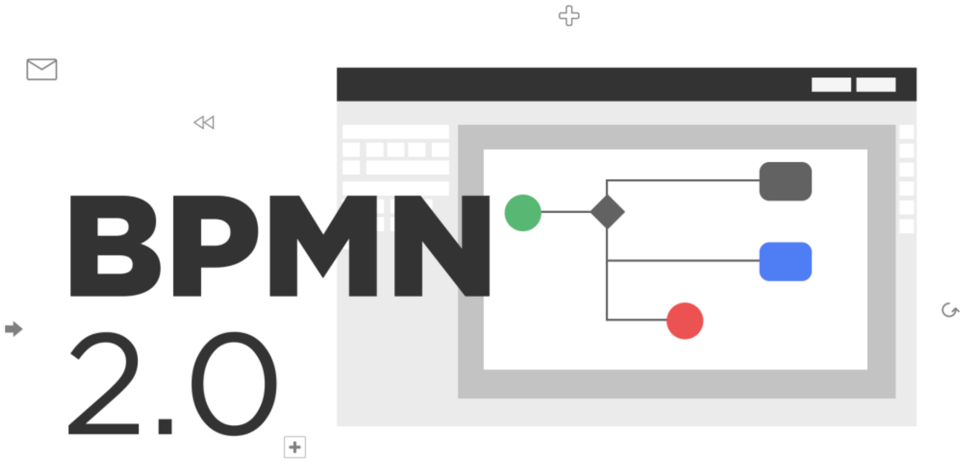
BPMN is flow-chart based notation for defining Business Processes

BPMN is an agreement between multiple modeLling tools vendors, who had their own notations, to use a single notation for the benefit of end-user understand and training

BPMN provides a mechanism to generate an executable. Business Process (BPEL) from the business level notation

A Business Process developed by a business analyst can be directly applied to a BPM engine instead of going through human interpretations and translations into other languages

Origin of BPMN



The Business Process Management Institute (BPMI- now a part of the OMG) develops BPML (an XML process execution language) and realizes need for a graphical representation

BPML was later replaced by BPEL as the target execution language

August, 2001.1 the Notation Working Group is formed.

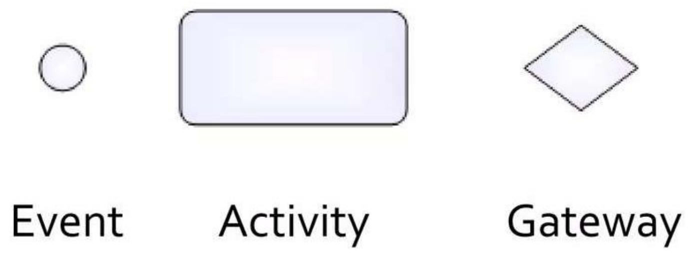
The group was composed of 35 companies, organizations, or individuals. BPMN 1.0

May, 2004, the BPMN 1.0 specification was released to the public

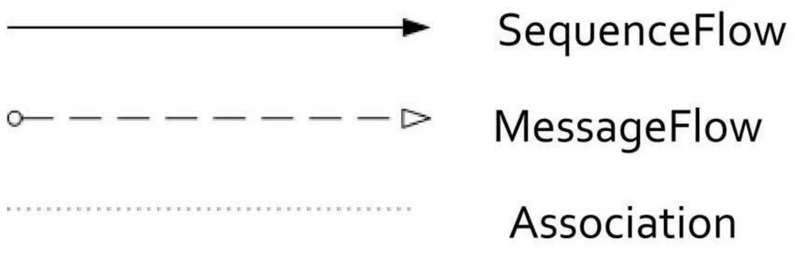
February, 2006, BPMN 1.0 was adopted as an OMG standard

Design Elements

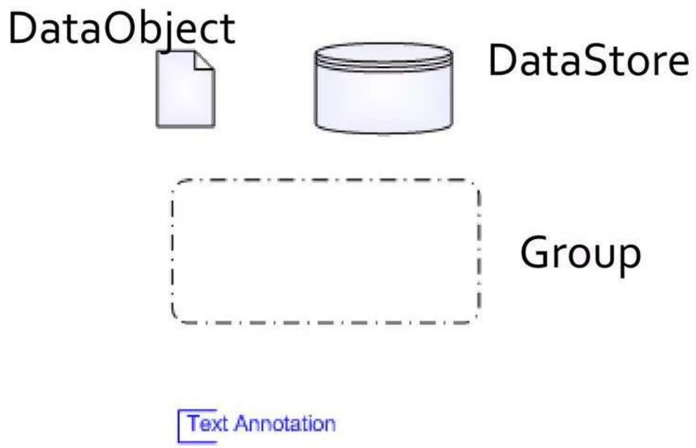
Flow Objects



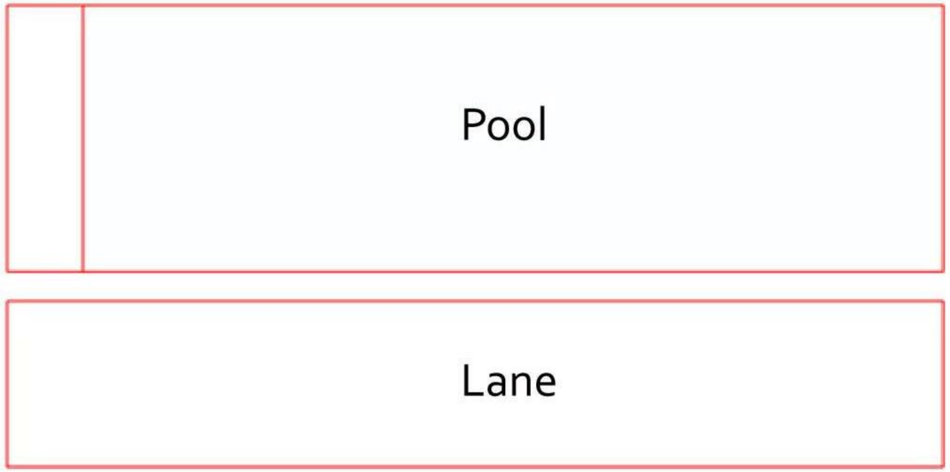
Connecting Objects



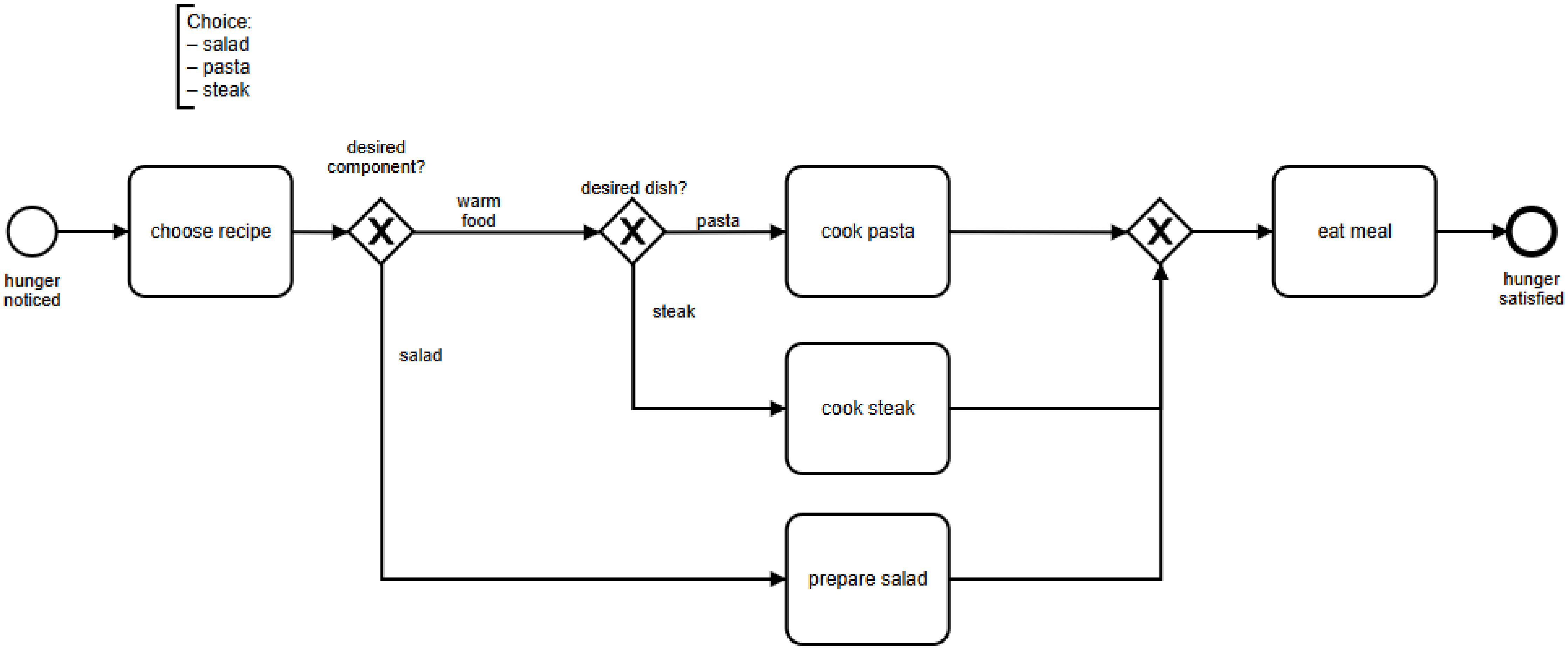
Data Objects & Artifacts



Swimlanes



Sample Process

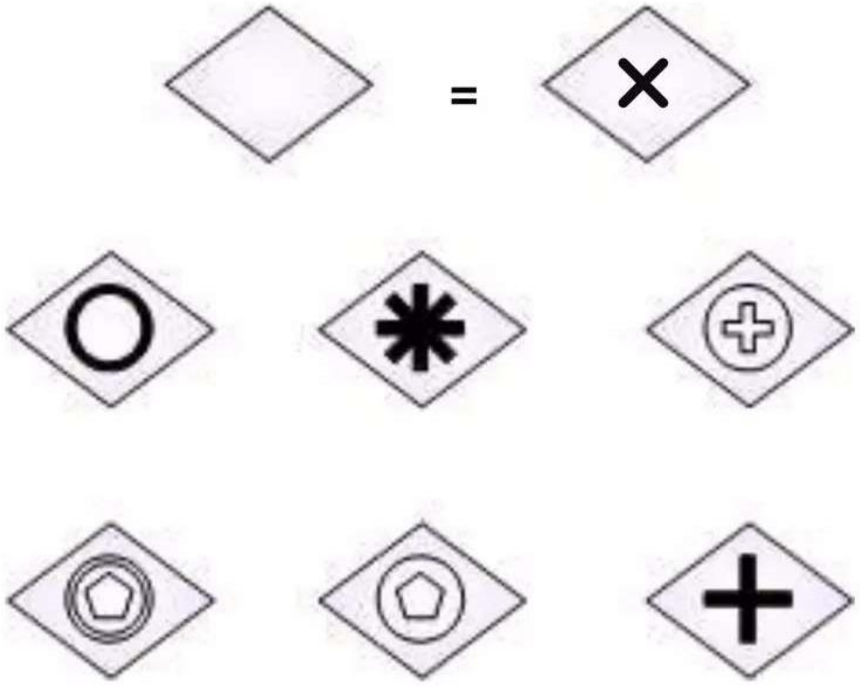


Gateway

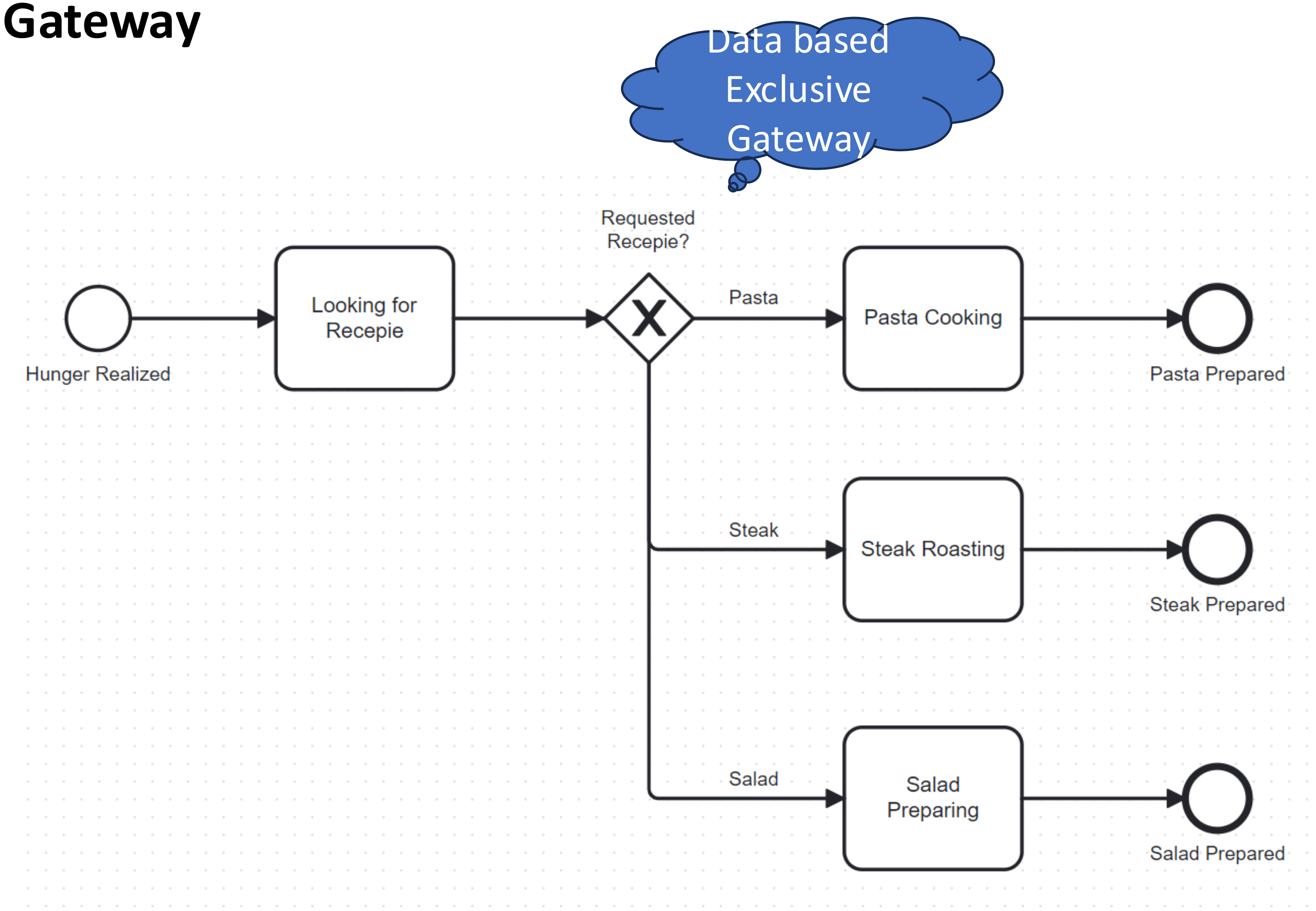
A **Gateway** is used to control the divergence and convergence of sequence flows in a Process or in a choreography.

Gateway will determine branching, forking, merging or joining.

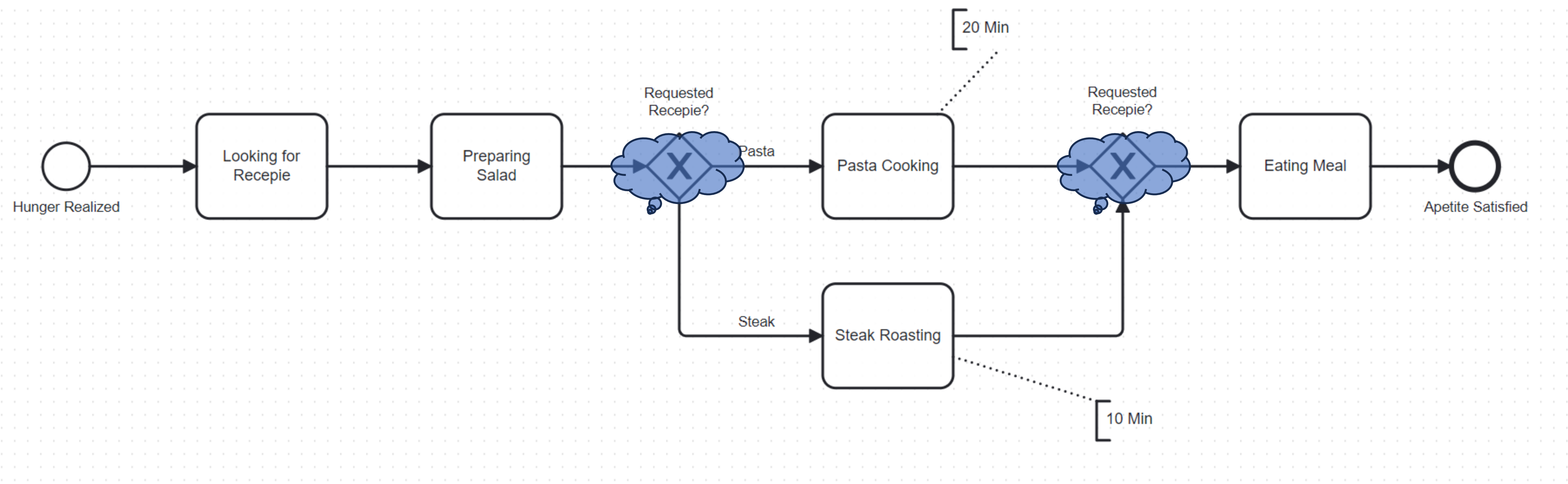
There's 7 kinds of gateways differed by its internal marker:
Exclusive, **Inclusive**, **Parallel**, **Complex**, **Event-based**, **Parallel Event-based** and **Exclusive Event-based**.



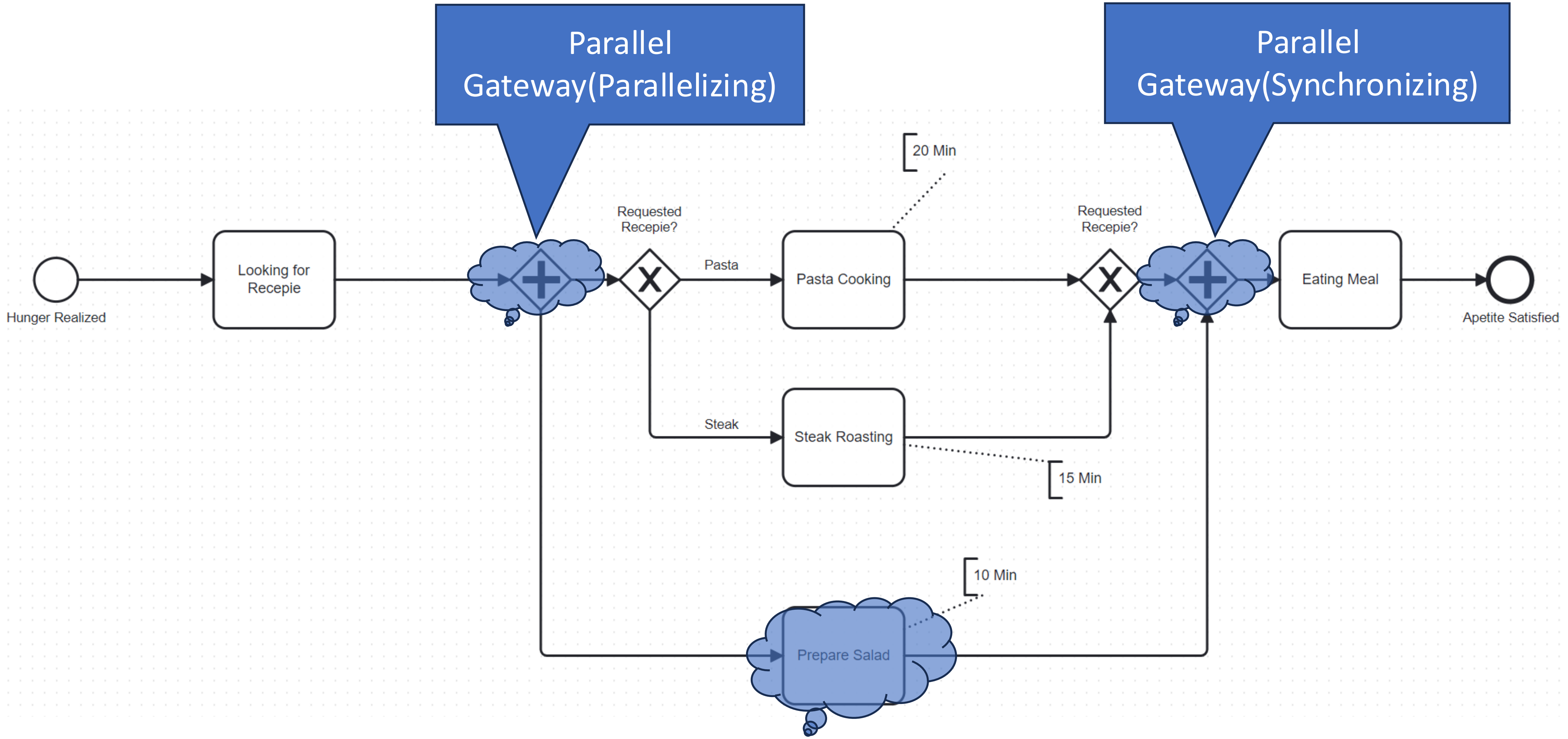
Exclusive (XOR) Gateway



Exclusive (XOR) Gateway Merging

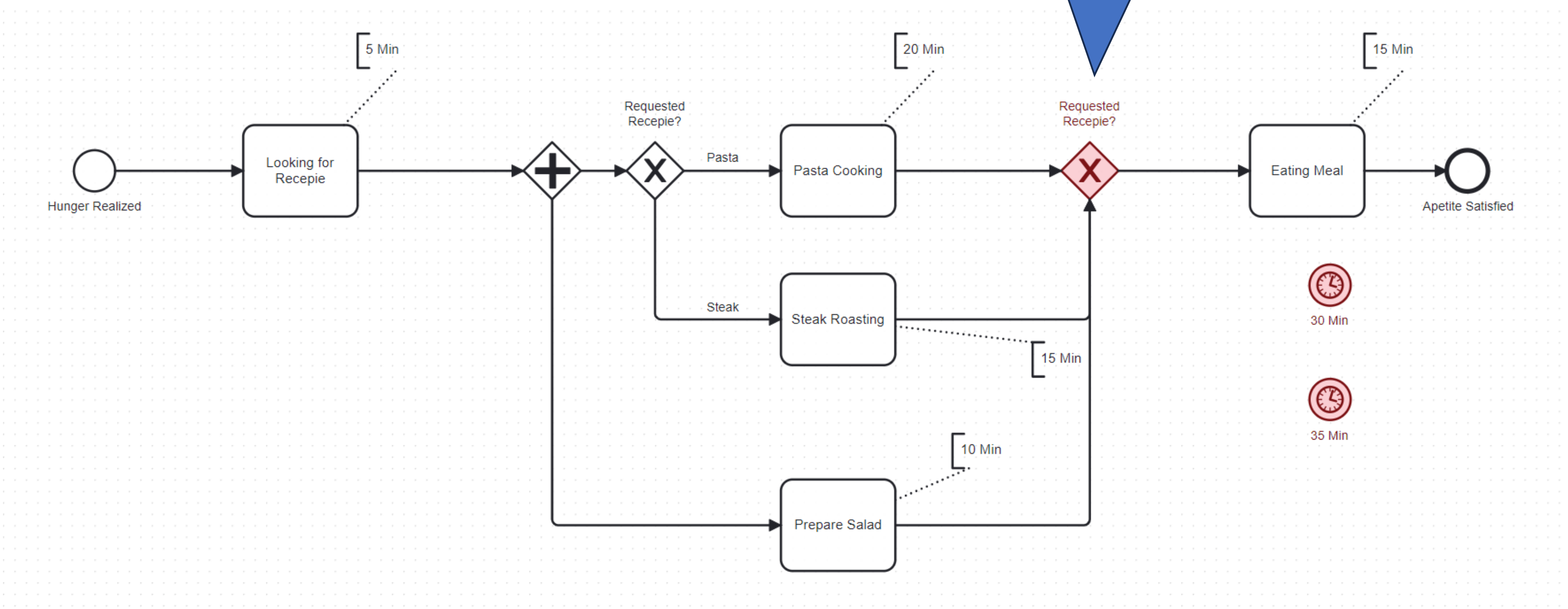


Parallel Gateway AND-Split & AND-Join

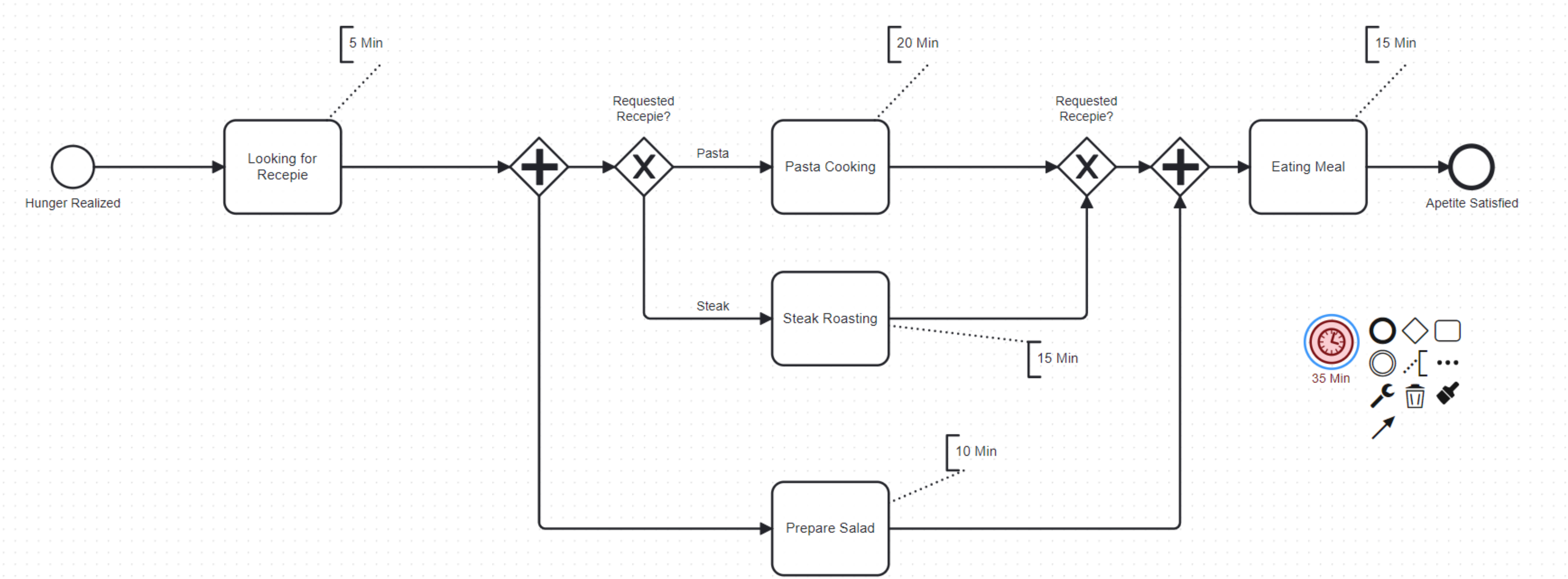


Understanding the token – When does it Finish ?

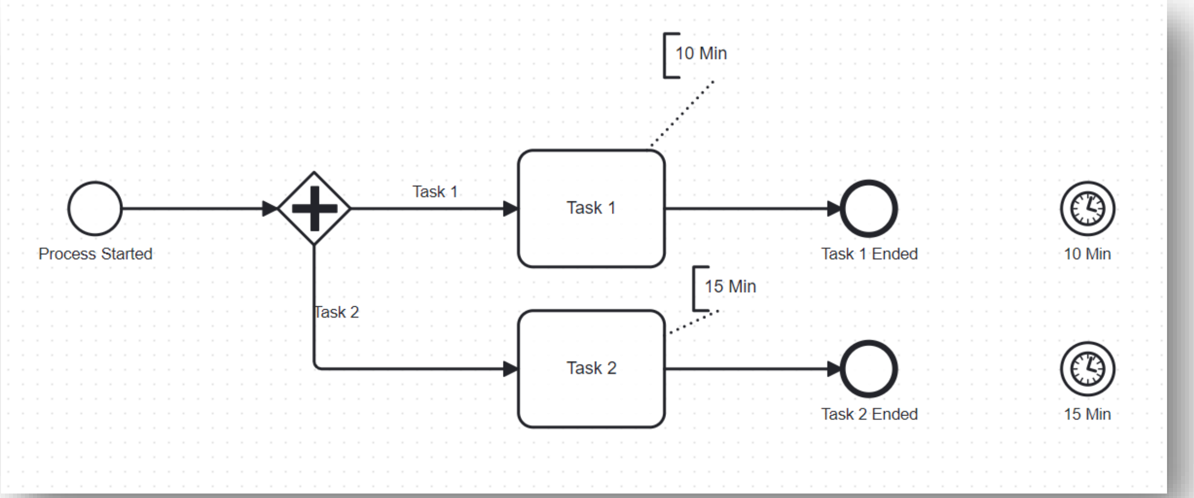
At this position the engine does not wait for second token

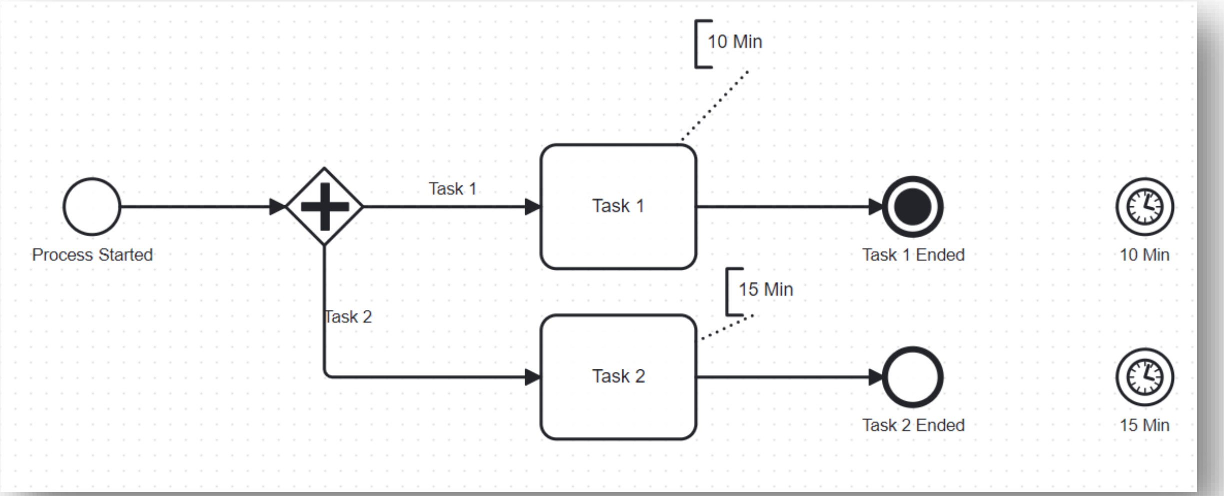


Understanding the token – When does it Finish ?



Understanding the token – When does it Finish ?



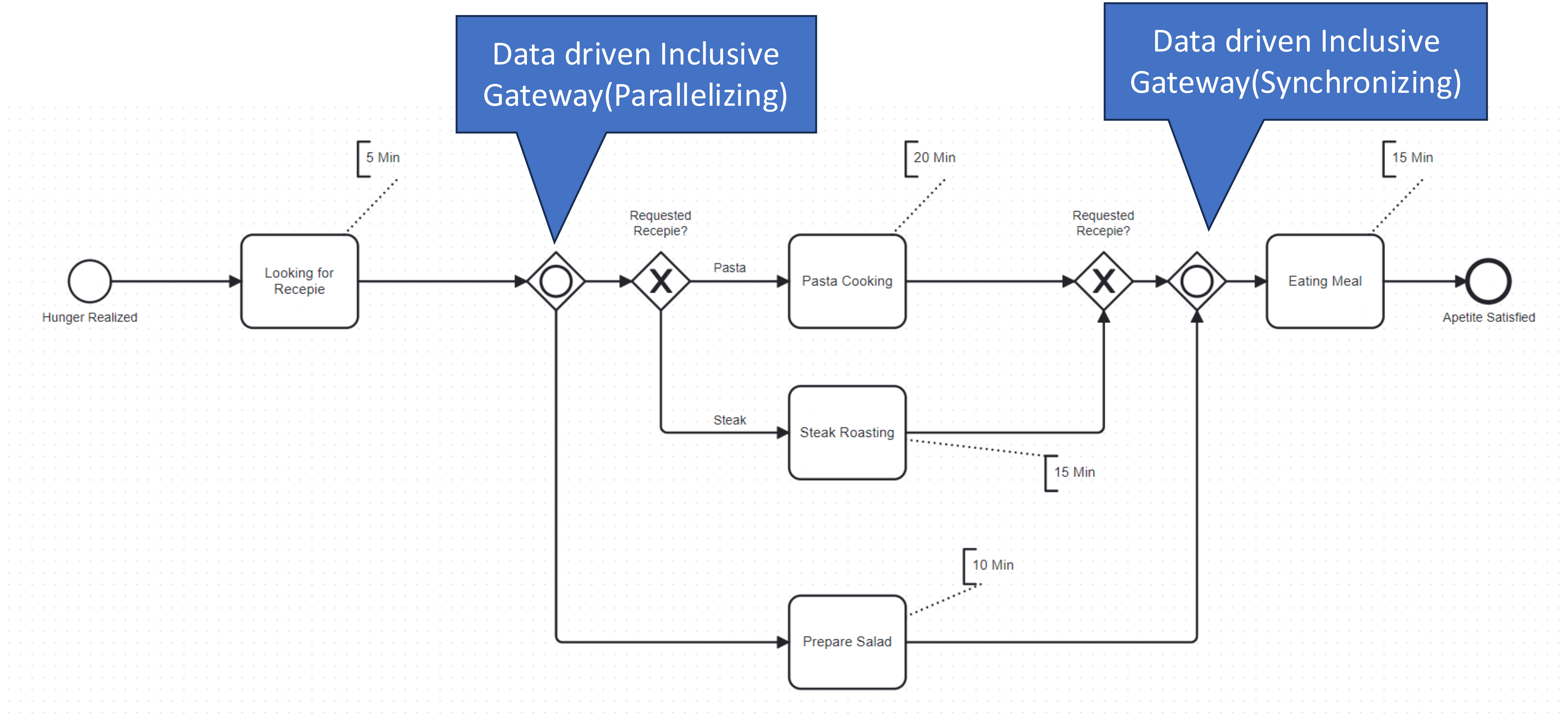


Data Driven Inclusive Gateway OR-Split & OR-Join

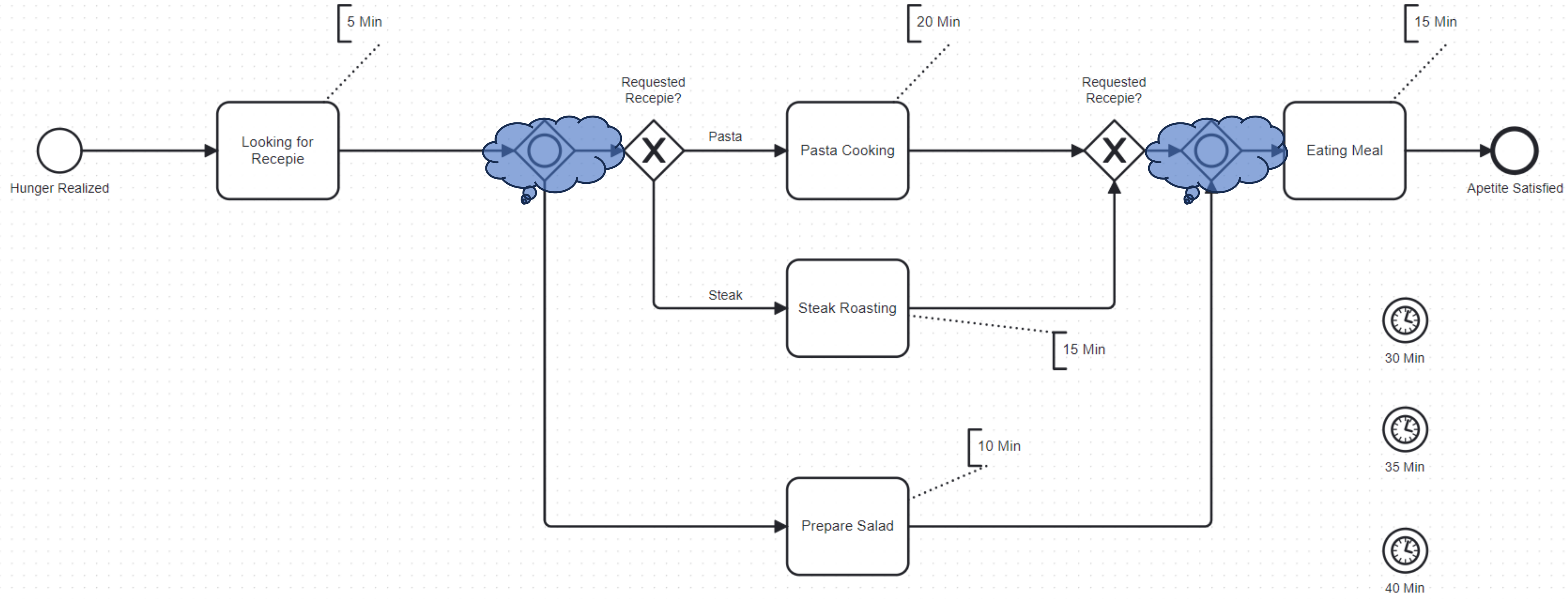
Requirement:

- Only one salad or
- One Salad and something substantial like pasta or steak
- Only Something Substantial

Data Driven Inclusive Gateway OR-Split & OR-Join

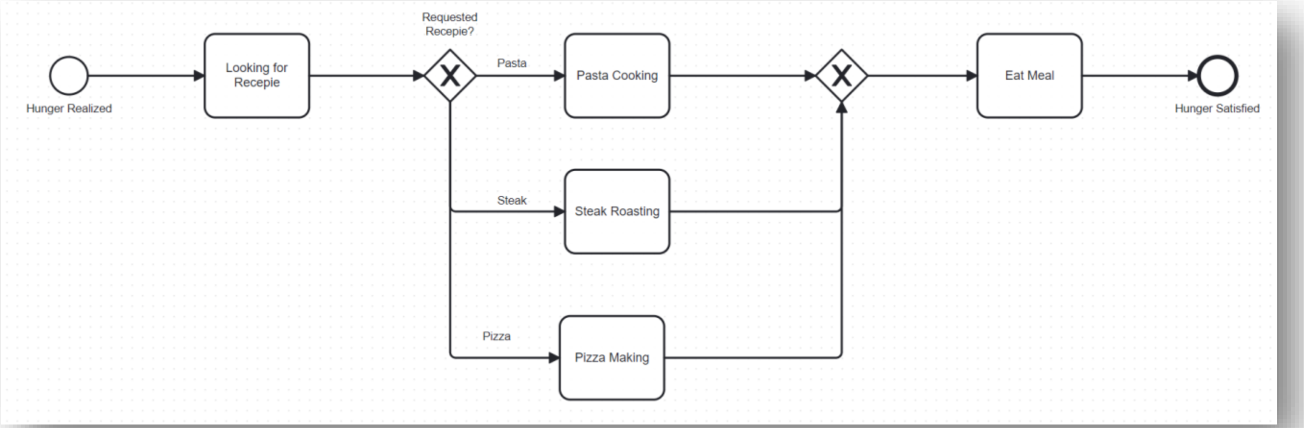


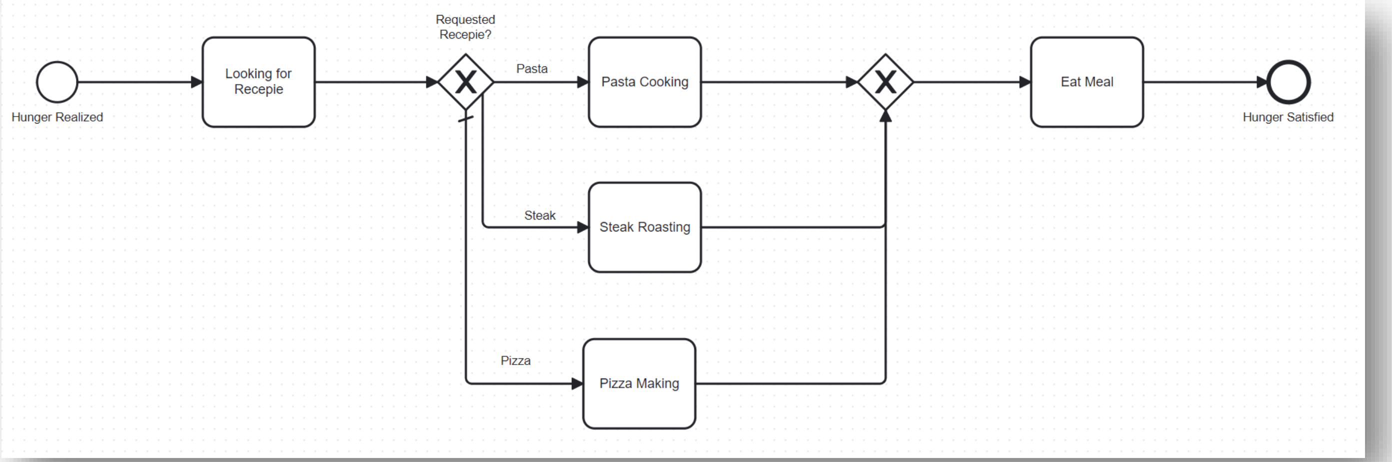
Data Driven Inclusive Gateway OR-Split & OR-Join



Standard Flow

What will happen if neither pasta or steak or pizza is chosen, This will be a case of deadlock, Default flow breaks the deadlock





BPMN Rules and Etiquette

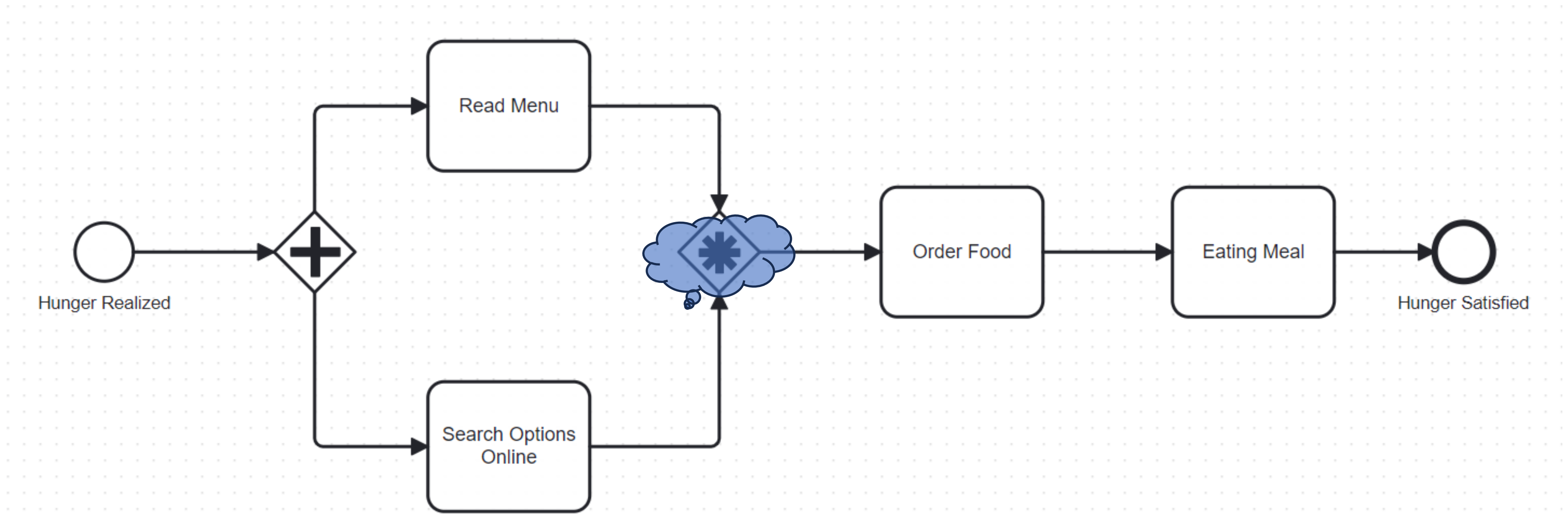
- BPMN does not specify what happens if no ending path is being executed
- There are no implicit sequence flows allowed => all sequence flows must be modelled
- Use a standard flow if there is a danger of a dead lock
- Always Draw models from left to right
- Do not cross flows

Complex Gateway

Exercise:

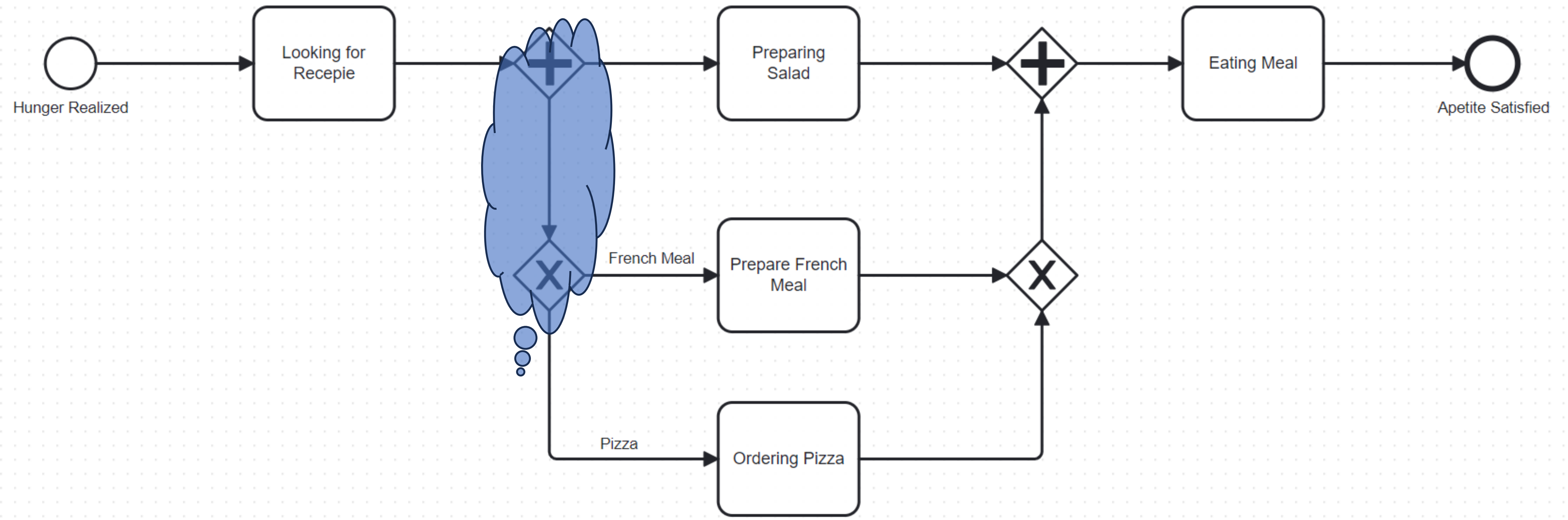
- We want to order pizza and are evaluating available menus and internet offers.
- As soon as one task is finished we want to order!

Complex Gateway

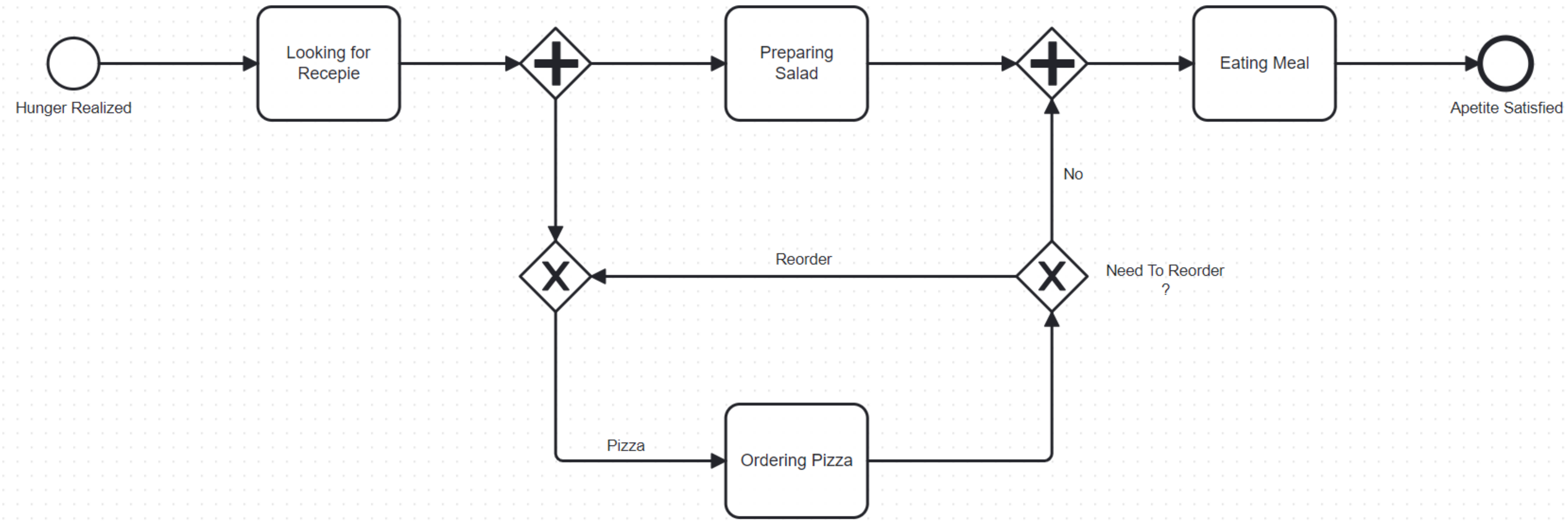


If several tokens are arriving at a complex gateway, the gateway will wait for a predefined number of tokens before the merged token will be routed to next task. (the number of tokens will be specified by a comment assigned to the complex gateway)

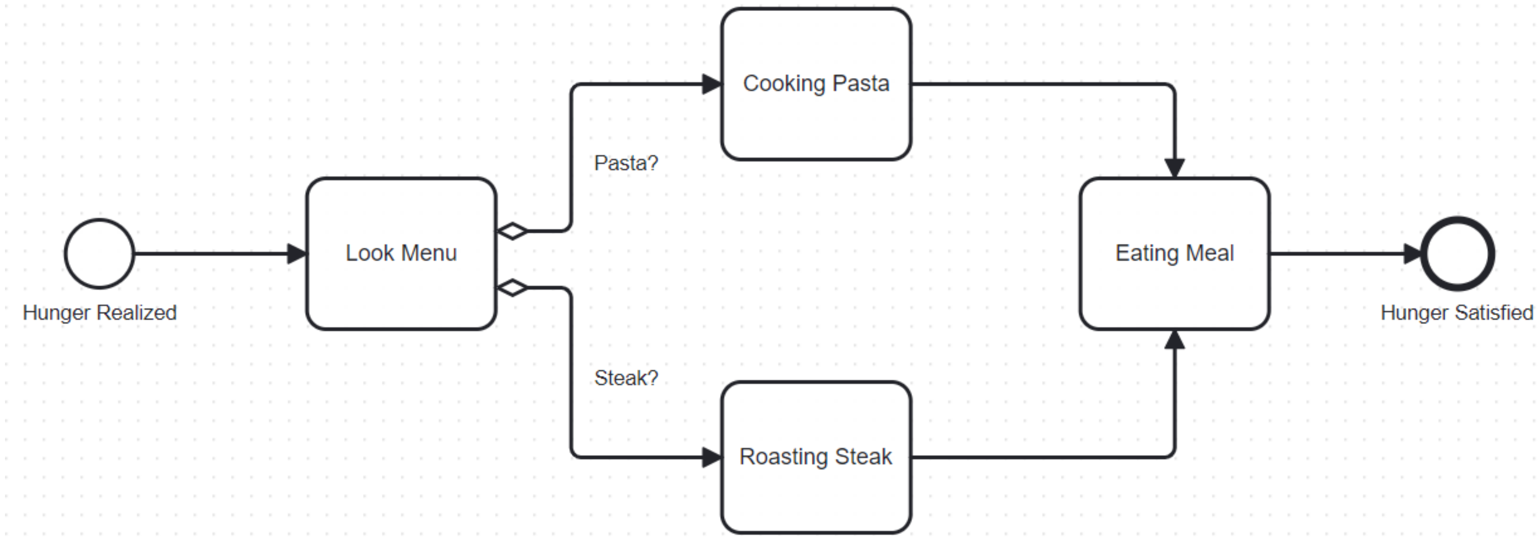
Combination of Gateways



Loop Back with Gateways



Conditional Flow



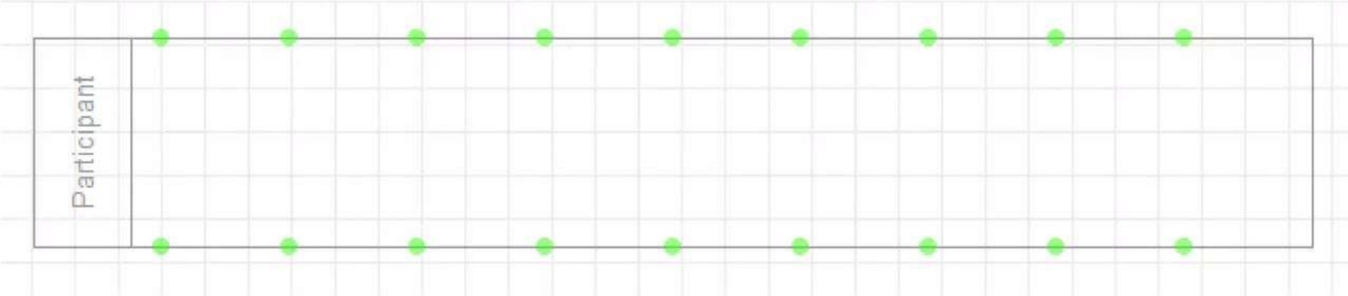
1. Without Join-Gateway it is not possible to model a synchronization!
2. Reduced possibilities for condition checks
3. Conditional flows following the semantic of OR-Splits

Swimlanes

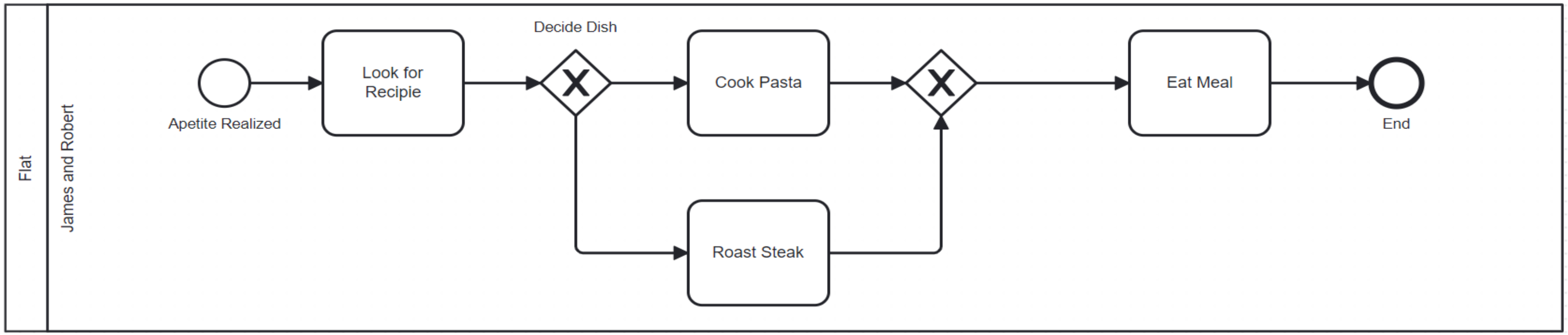
Pool or Laneset is an element representing a process into an organization or company.

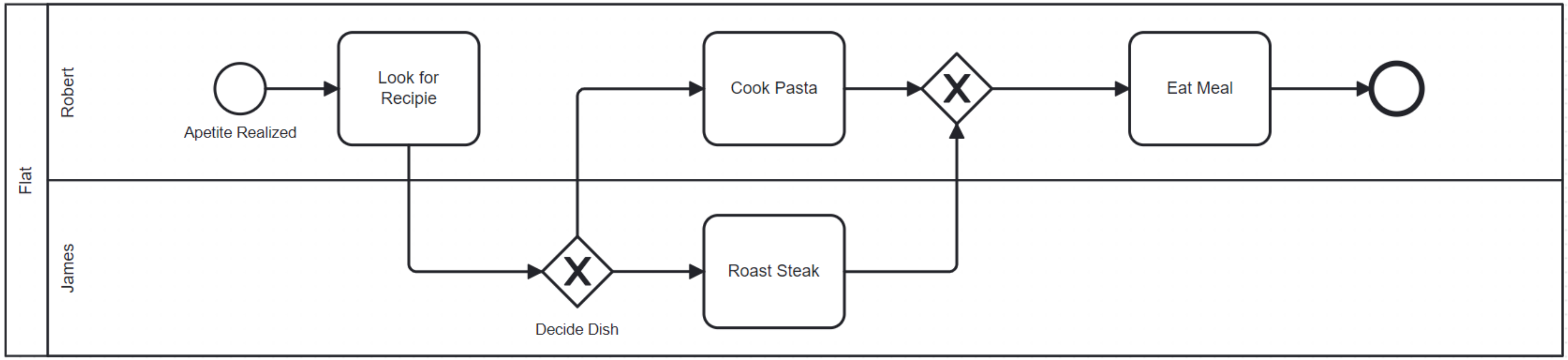
Lane is a representation of an area or department of the company. Some times can represent a role into a process scope.

Participant or Empty Pool is a representation of a process or entity that does not have any action within the process.

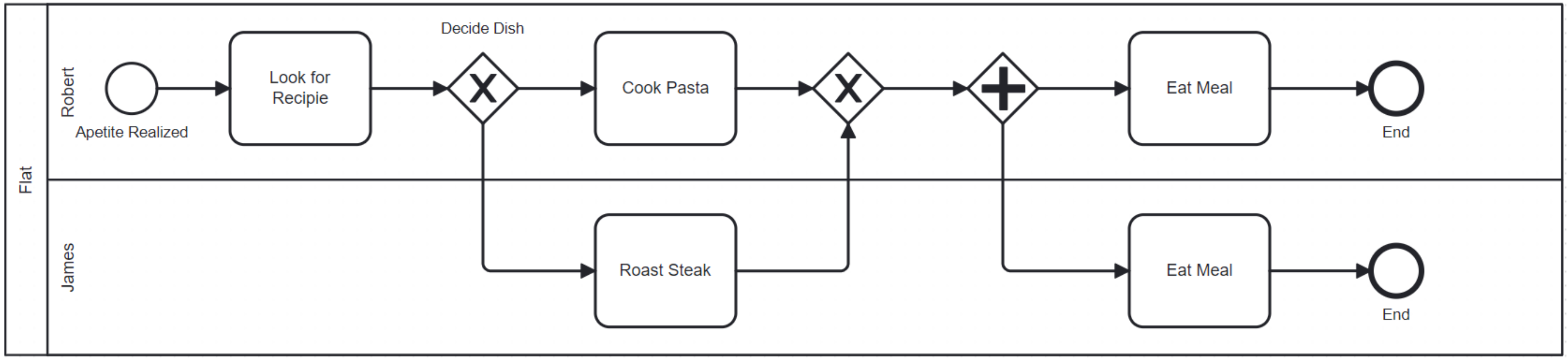


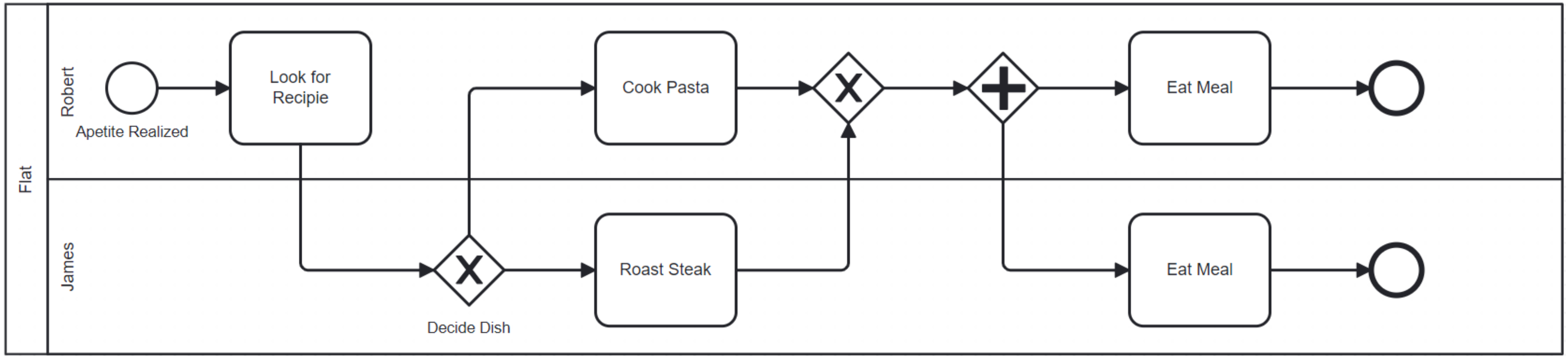
Swimlanes


Flat
James and Robert
Apetite Realized
Look for Recipie
Decide Dish
Cook Pasta
Roast Steak
Eat Meal
End

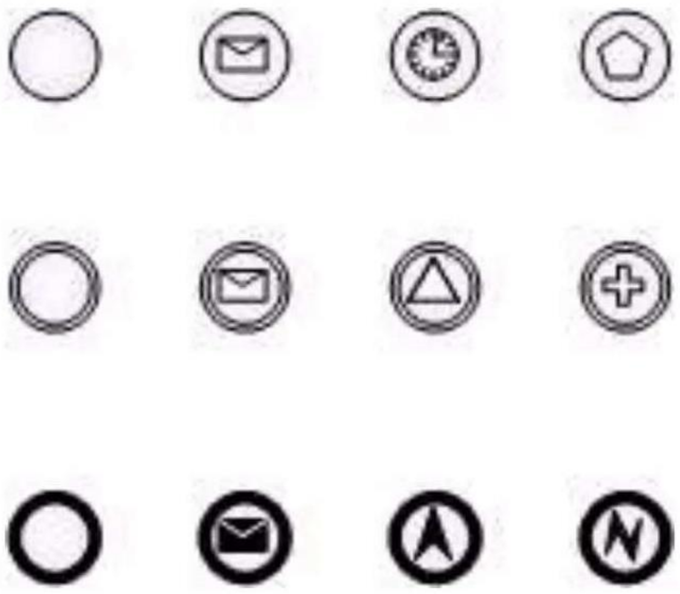

Flat
Robert
James
Apetite Realized
Look for Recipie
Cook Pasta
Eat Meal
Decide Dish
Roast Steak

Swimlanes


Flat
Robert
James
Apetite Realized
Look for Recipie
Decide Dish
Cook Pasta
Roast Steak
Eat Meal
Eat Meal
End
End


Flat
Robert
James
Apetite Realized
Look for Recipie
Decide Dish
Cook Pasta
Roast Steak
Eat Meal
Eat Meal

Event



An **Event** is something that “happens” during the course of a Process or a Choreography. An Event affects the flow of the model and usually have a cause (Trigger) or an impact (Result).

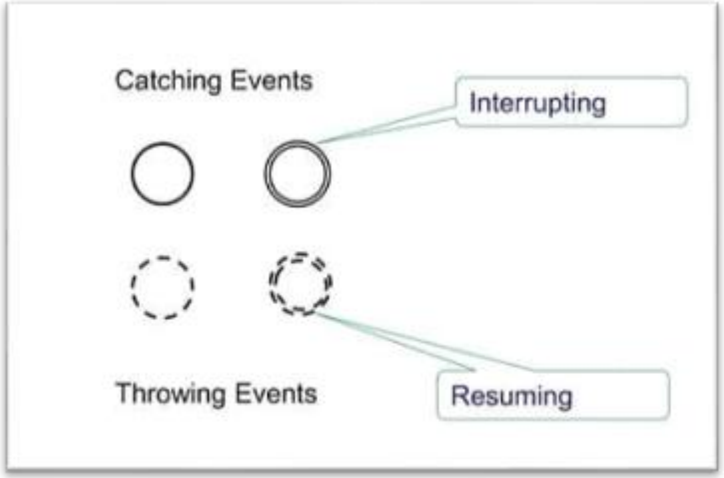
Event graphical representation is a circle.

There’s 3 types of events: **Start** Events, **Intermediate** Events and **End** Event.

Intermediate Events can be used in regular process diagrams and can be used as **Boundary** Events attached to an activity.

Start Events and Intermediate Events can be **Interrupting** and **Non-interrupting**.

Broad Category



Catching events are events with a defined trigger. We consider that they take place once the trigger has activated or fired. As an intellectual construct, that is relatively intricate, so we simplify by calling them catching events. The point is that these events influence the course of the process and therefore must be modelled. Catching events may result in:

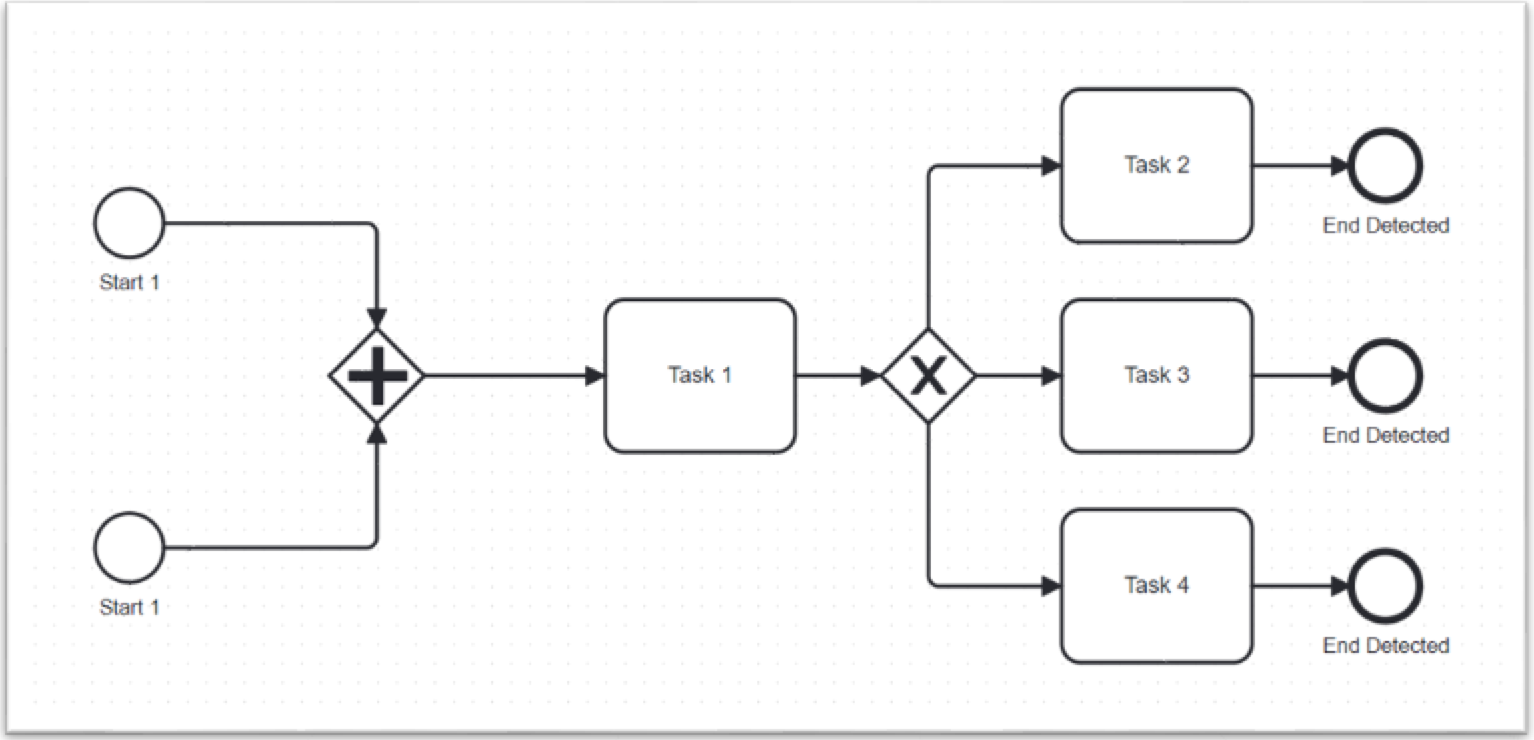
- The process starting
- The process or a process path continuing
- The task currently processed or the sub-process being cancelled
- Another process path being used while a task or a sub-process executes

Throwing events are assumed by BPMN to trigger themselves instead of reacting to a trigger. You could say that they are active compared to passive catching events. We call them throwing events for short, because the process triggers them. Throwing events can be:

- Triggered during the process
- Triggered at the end of the process

Wrong Event Usage

Two start events are throwing two different process universal ids = two process instances



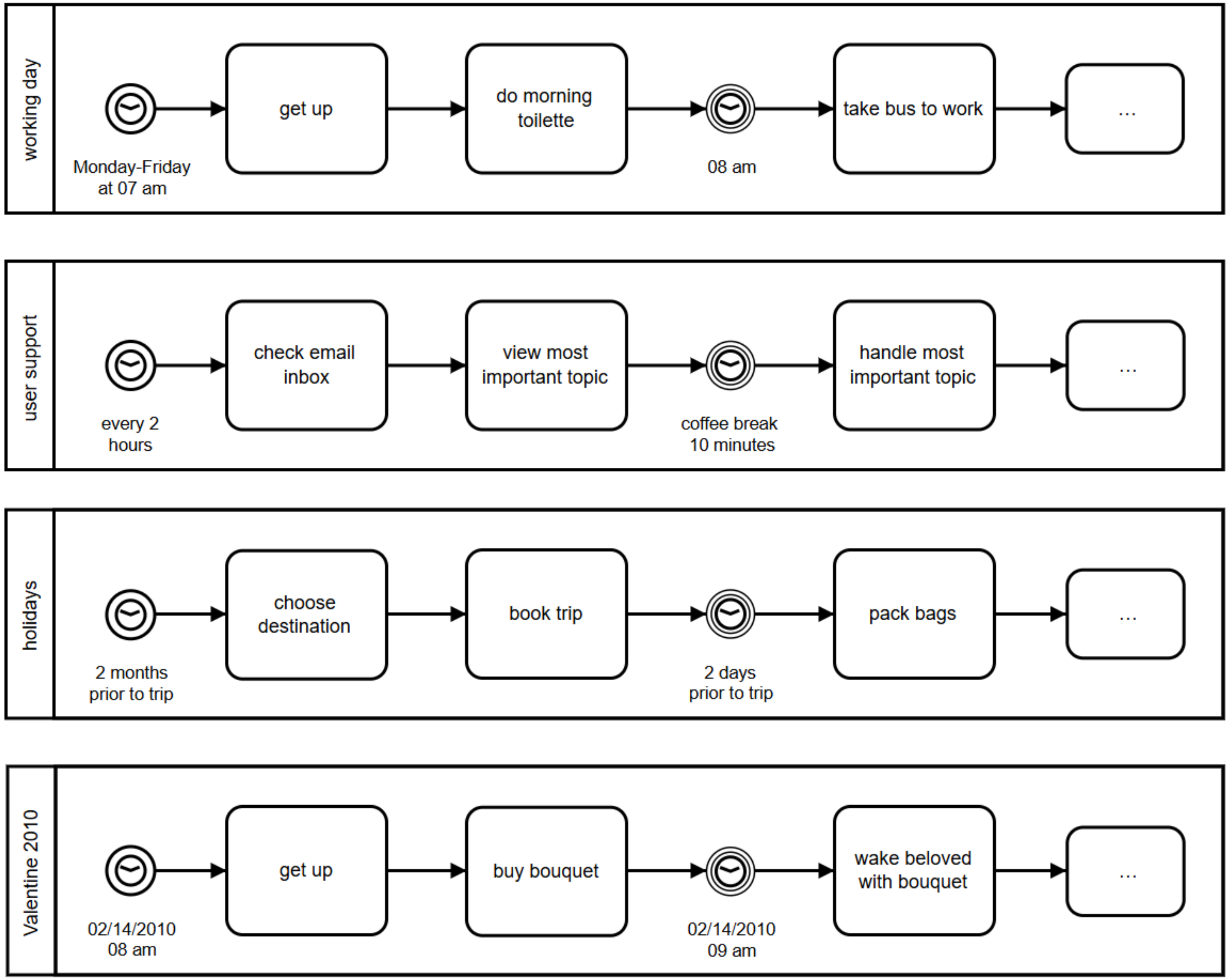
Start Event

Start Events indicate where a Process will begin

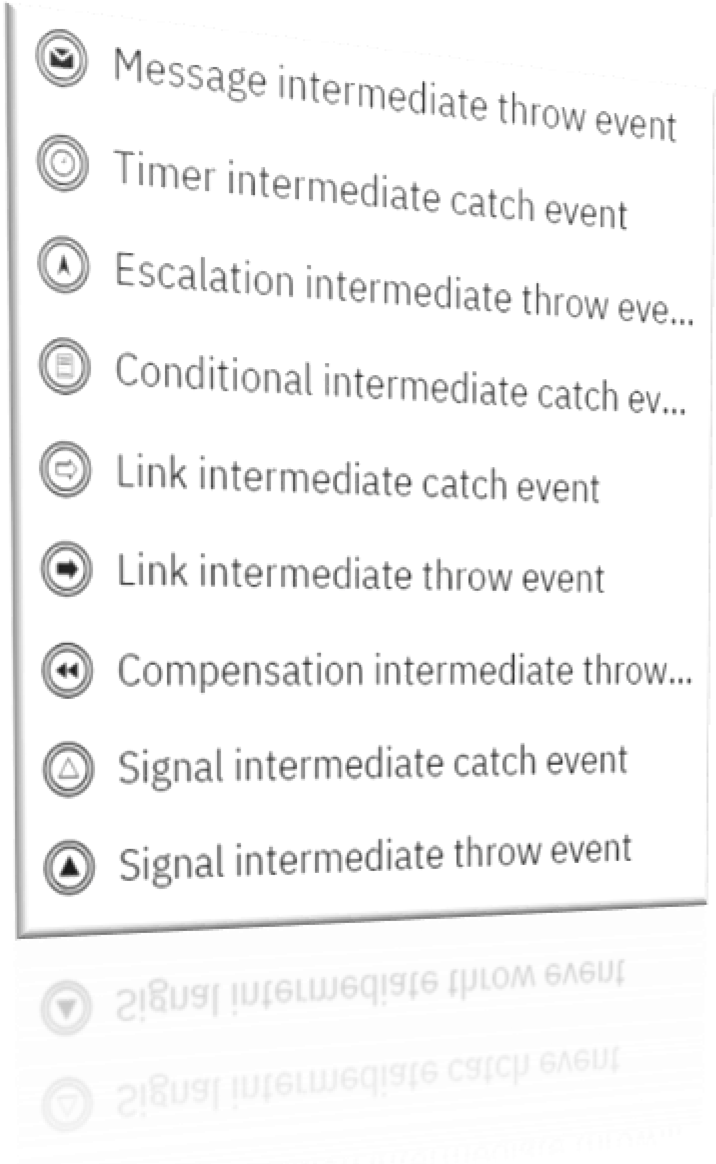
There are different "Triggers" that indicate the specific circumstances that start the Process

- None Start Events are used to mark the start of Sub-Processes or when the start is undefined
- The Link Start Event will be removed in the next version of BPMN
- Any one of the Triggers included in a Multiple Start Event will start the Process

Start Event Example



Intermediate Event



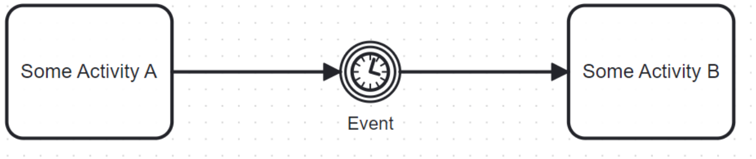
Intermediate Events occur after a process has been started and before a process is ended

There are different "Triggers" that indicate the specific circumstances of the Event

They can be placed in the normal flow of the Process or attached to the boundary of an activity

Intermediate Event

Normal Flow

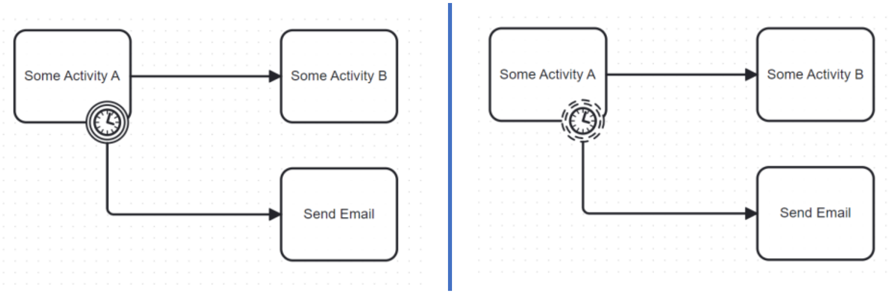


Events that are placed within the process flow represent things that happen during the normal operations of the process

They can represent the response to the Event (i.e.. the receipt of a message)

They can represent the creation of the Event (i.e., the sending of a message)

Boundary Attached

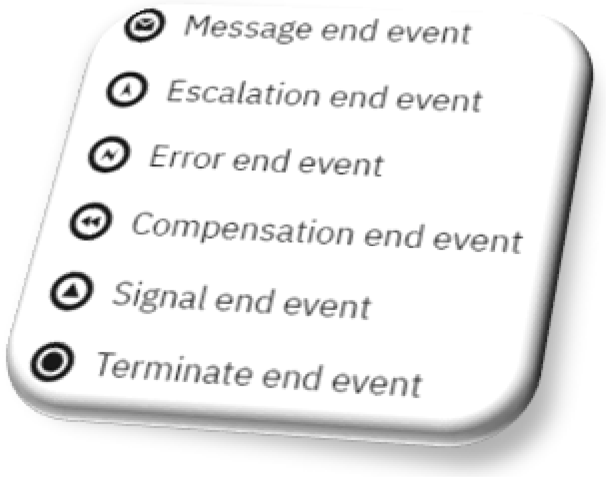


Events that are attached to the boundary of an activity indicate that the activity should be interrupted when the Event is triggered

They can be attached to either Tasks or Sub-Processes

They are used for error handling, exception handling, and compensation

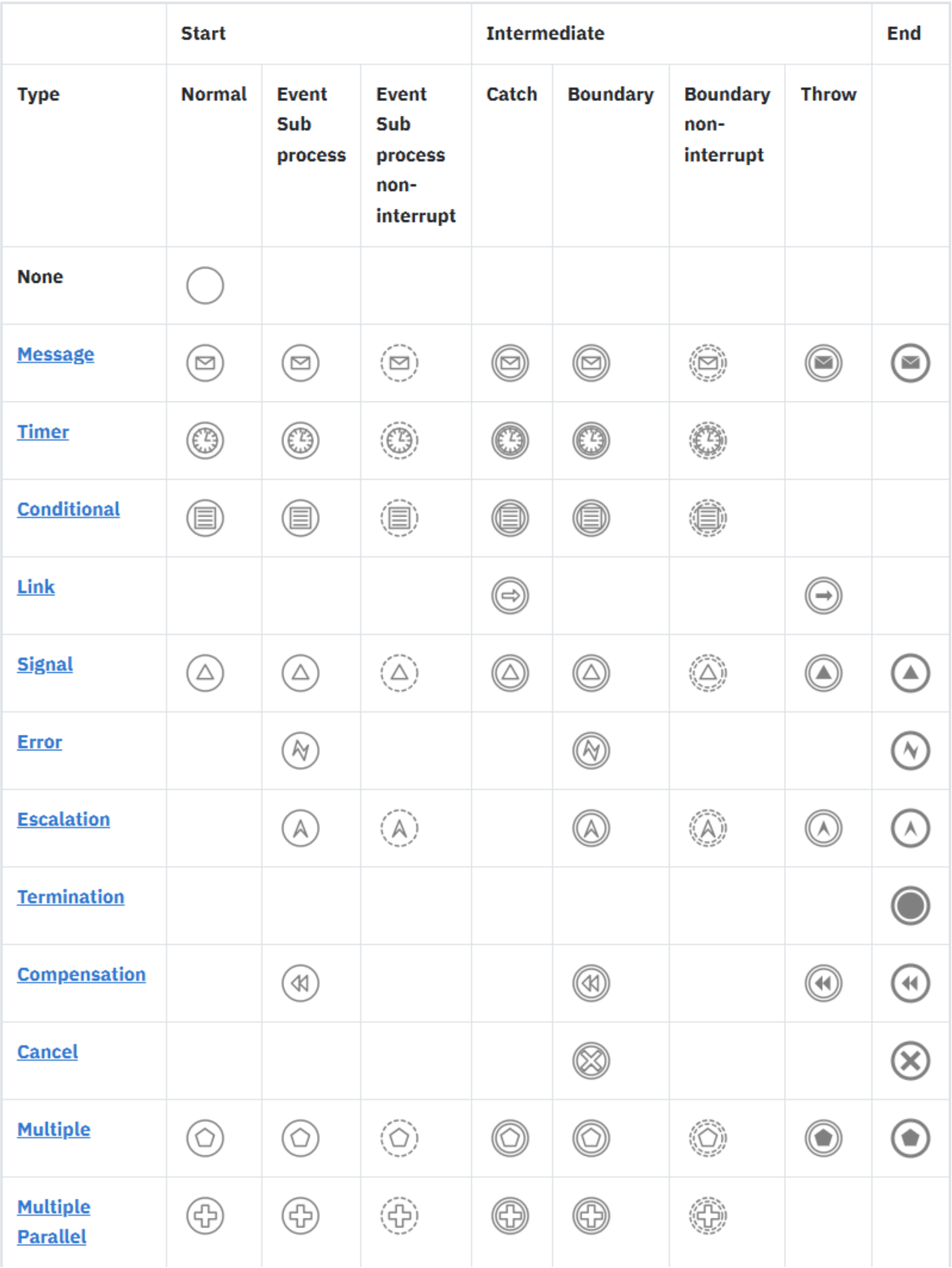
End Event



End Events indicates where a process will end

There are different "Results" that indicate the specific circumstances that end the Process

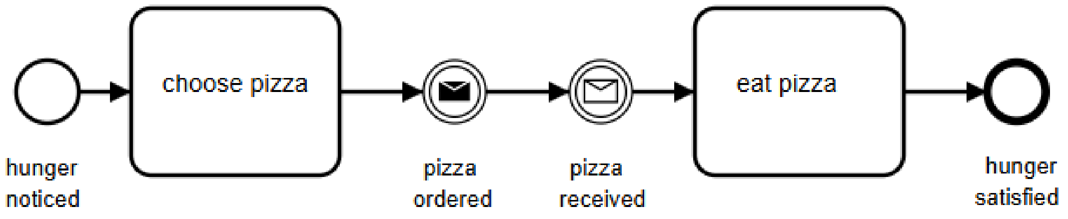
Summary Sheet - Events

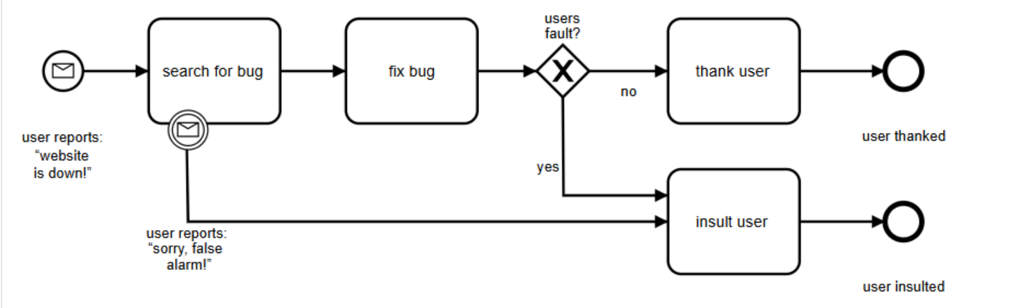


	Start			Intermediate				End
Type	Normal	Event Sub process	Event Sub process non-interrupt	Catch	Boundary	Boundary non-interrupt	Throw	
None								
Message								
Timer								
Conditional								
Link								
Signal								
Error								
Escalation								
Termination								
Compensation								
Cancel								
Multiple								
Multiple Parallel								

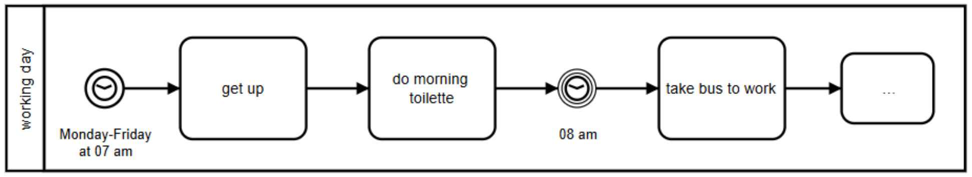
Event Examples

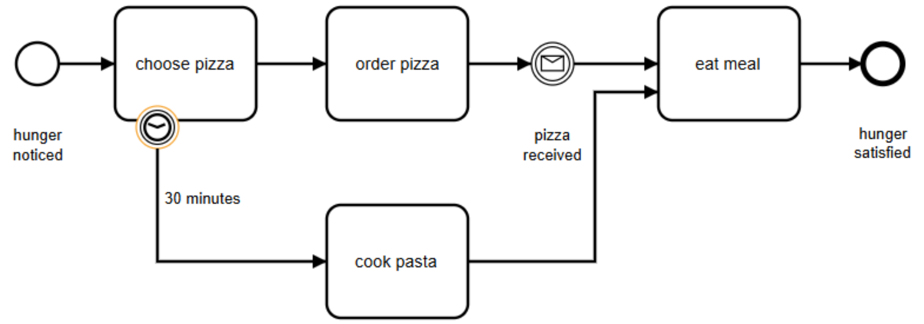
Message





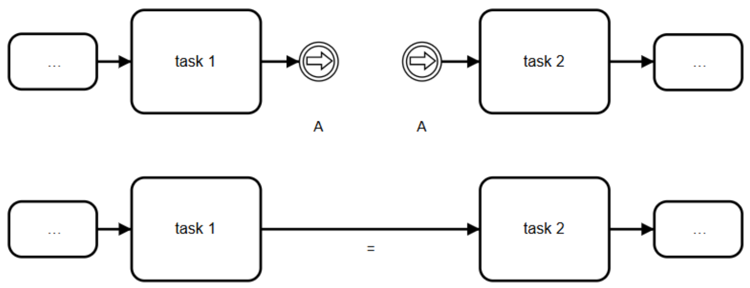
Timer





Intermediate Event

Link

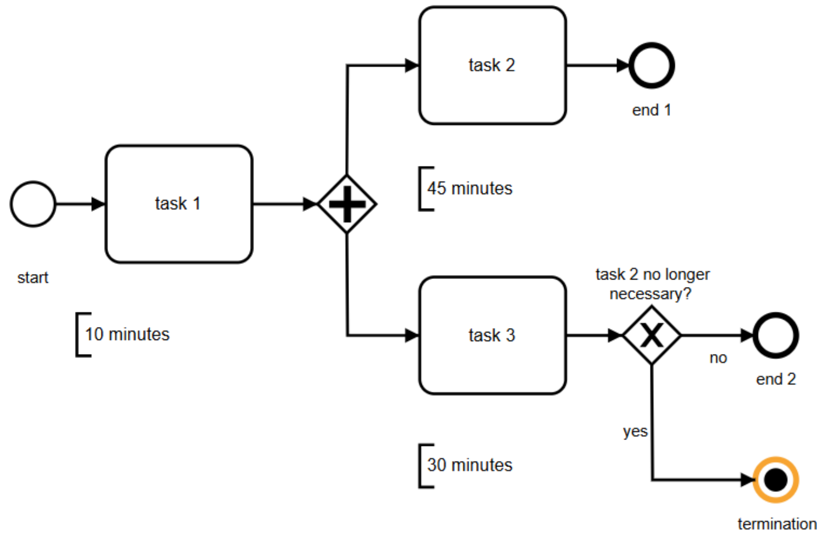


Link events can be very useful if:

•You have to distribute a process diagram across several pages. Links orient the reader from one page to the next.

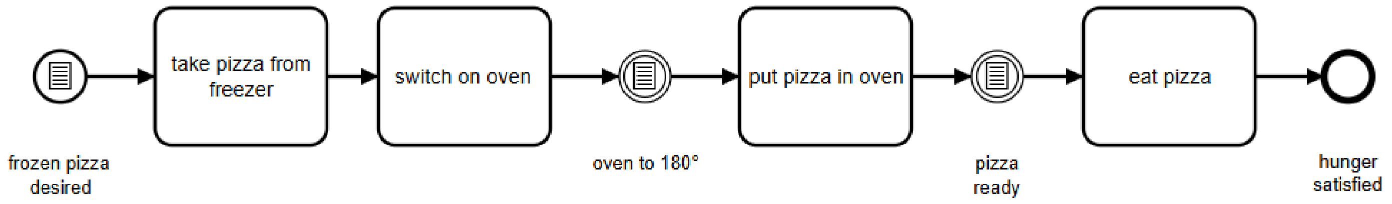
•You draw comprehensive process diagrams with many sequence flows. Links help avoid what otherwise might look like a “spaghetti” diagram.

Terminate



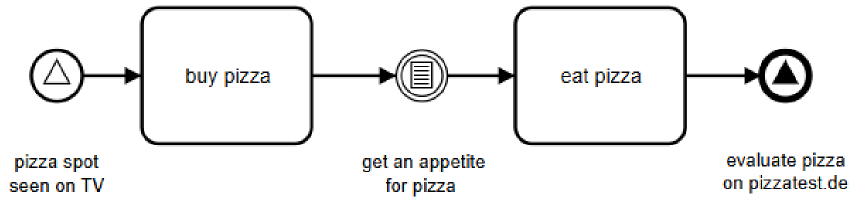
Event Examples

Conditional



Sometimes we only want a process to start or to continue if a certain condition is true. Anything can be a condition, and conditions are independent of processes, which is why the condition (like the timer event) can only exist as a catching event. A process cannot therefore conditional event trigger a conditional event.

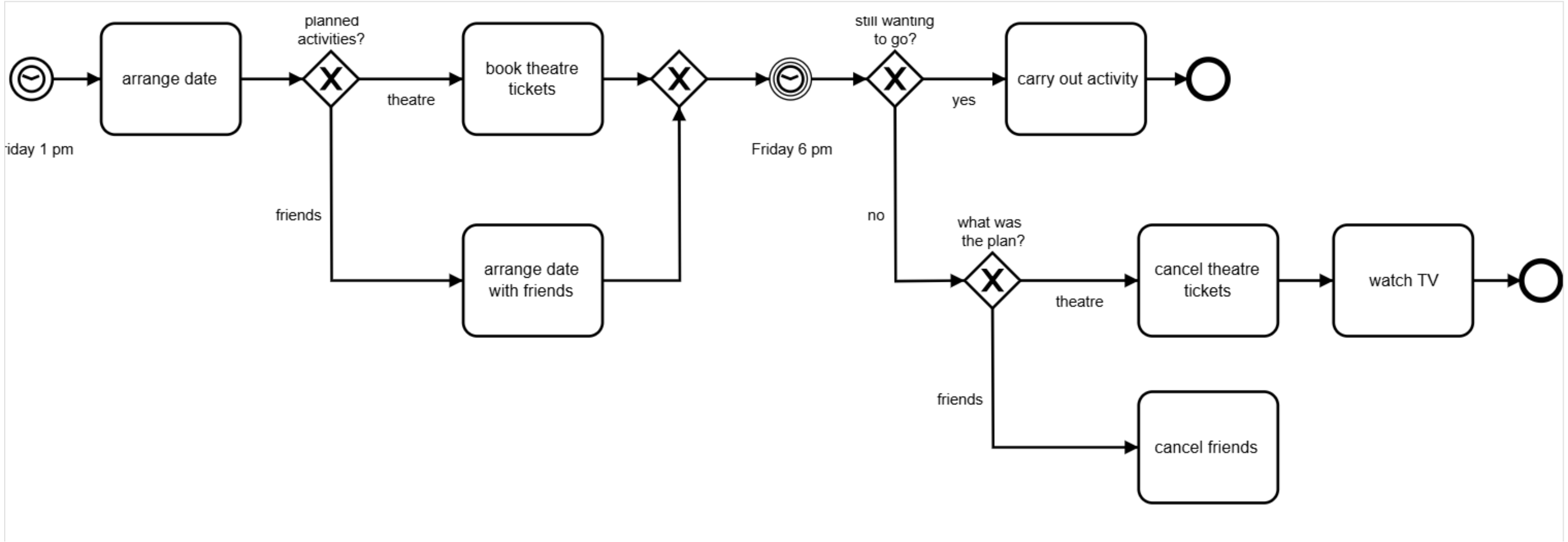
Signal



Signals are similar to messages, which is why you can model them in BPMN as events just as you can with messages. The symbol for a signal is a triangle. The essential difference between a signal and a message is that that latter is always addressed to a specific recipient. (An e-mail contains the e-mail address of the recipient, a call starts with dialing the telephone number, and so on.) In contrast, a signal is more like a newspaper advertisement or a television commercial. It is relatively undirected. Anyone who receives the signal and wants to react may do so.

Event Examples

Compensation



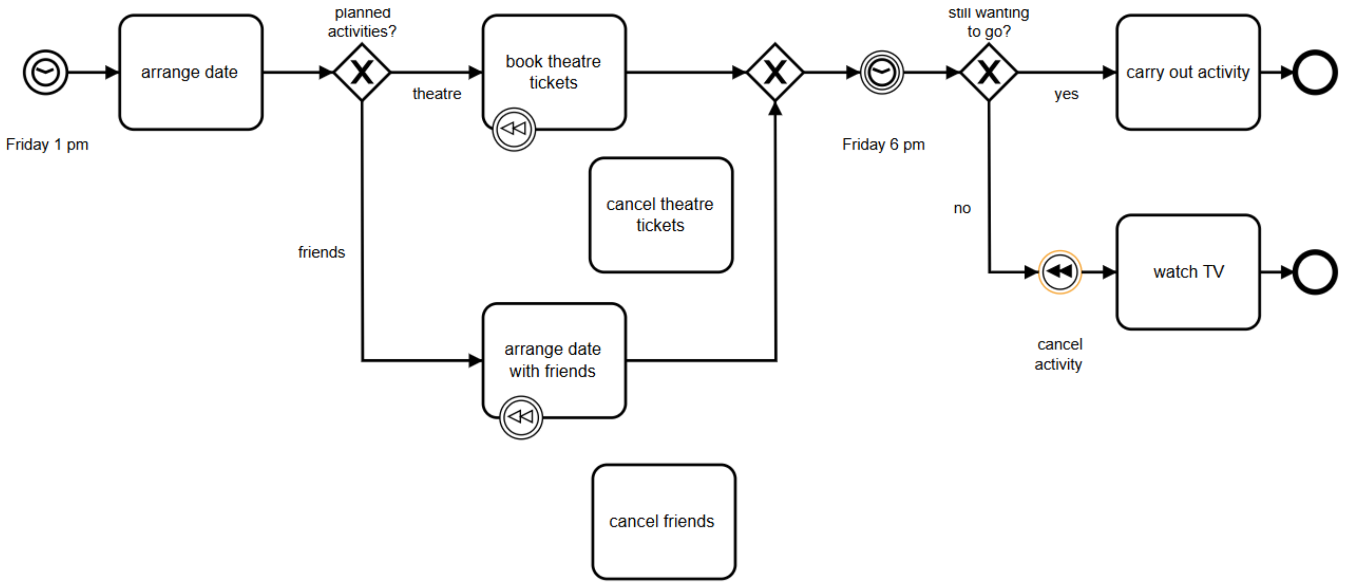
We execute tasks in our processes that sometimes have to be canceled later under certain circumstances.

Typical examples are:

•Booking a train or airline ticket
•Reserving a rental car
•Charging a credit card
•Commissioning a service provider

Event Examples

Compensation



There are special rules for handling compensations:

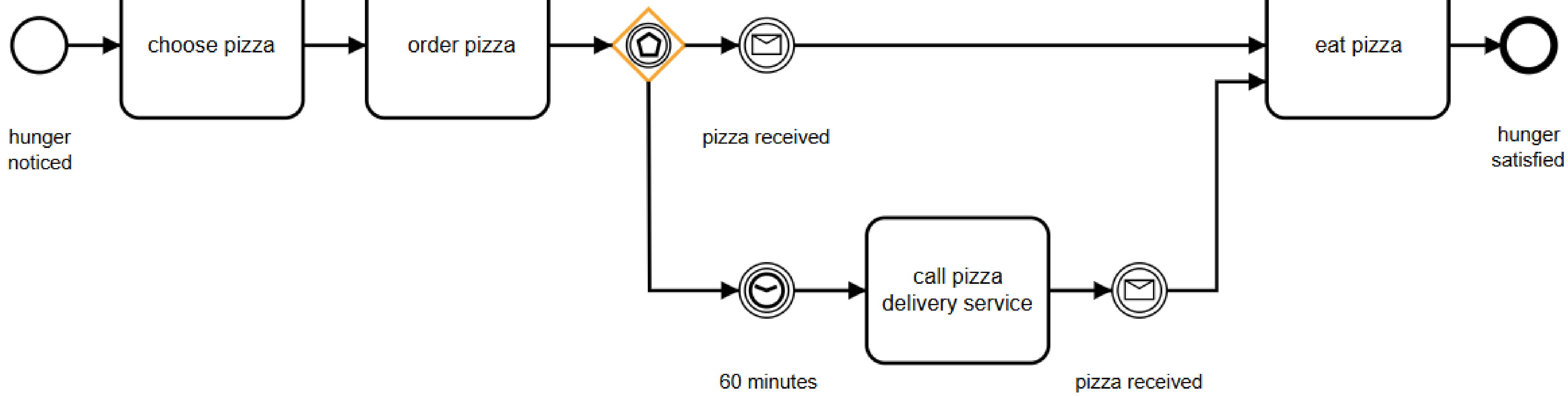
•Throwing compensations refer to their own processes, so the event is effective within the pool. This shows how this event type differs from a throwing message event.

•Other attached events can take effect only while the activities to which they are attached remain active. In contrast, an attached compensation takes effect only if the process triggers a compensation **and** the activity to which the compensation is attached successfully completed.

•Attached compensation events connect to compensation tasks through associations, and **not** through sequence flows, which would otherwise be common usage. BPMN thus emphasizes that compensations are beyond the regular process sequence; executing one is an exception.

Event Based Gateway

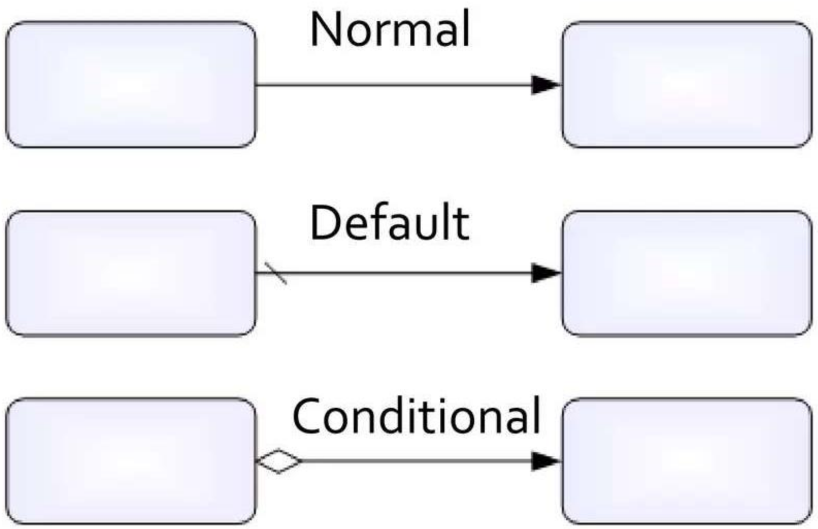
We learned about the exclusive data-based (XOR) gateway option as a way to use different paths with regard to the data being processed. Users of other process notations recognize this type of branching, but BPMN gives us another way to design process paths: the event-based gateway – event gateway, for short. This gateway does not route based on data, but rather by which event takes place next. To understand the benefit, consider the process shown below: We order pizza and wait for it to be delivered. We can eat only after we receive the pizza, but what if the pizza doesn't arrive after 60 minutes? We'll make an anxious phone call, that's what! We can model this with the event gateway:



Connection Objects

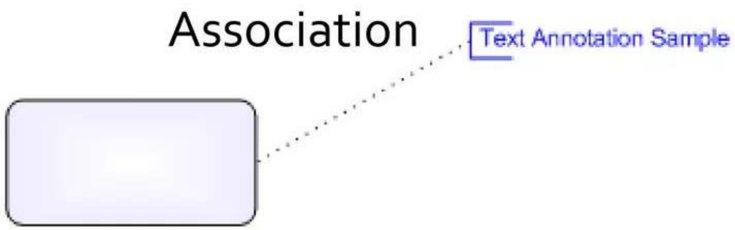
There's 6 types or connection objects. All of them are represented for a line.

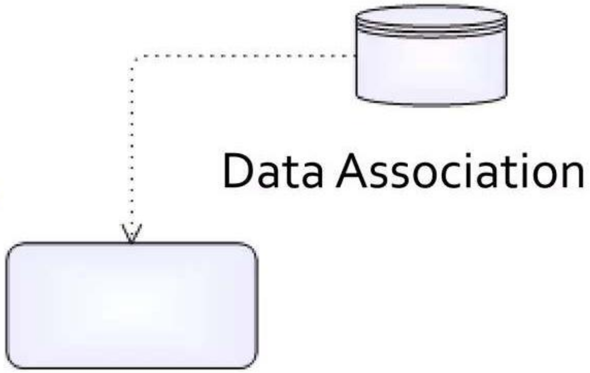
Sequence Flow can be **Normal**, **Default** and **Conditional**, and always have direction, source and target.

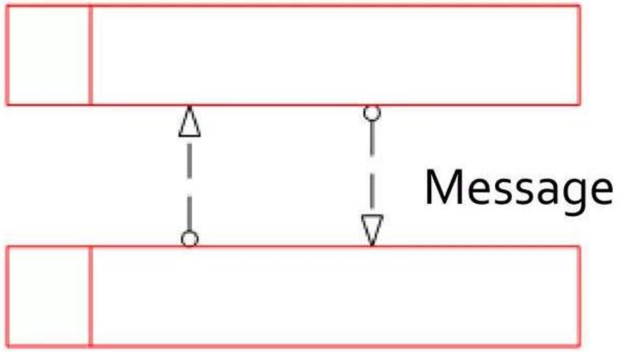


Message Flows are a type of connection object that is used to represent collaboration between two process.

Data Association is a line between a Data Object and An element.

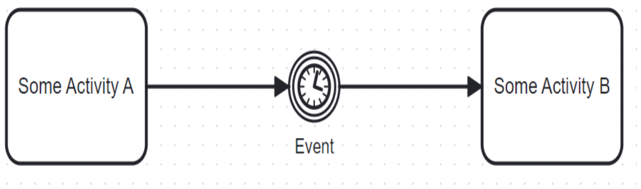






Connection Objects

Sequence Flow

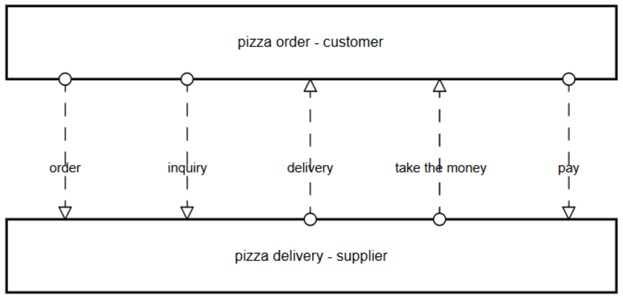


A Sequence Flow is used to show the order that activities will be performed in a Process

The source and target must be one of the following objects:

- Events, Activities, and Gateways
- A Sequence Flow cannot cross a Sub-Process boundary or a Pool boundary

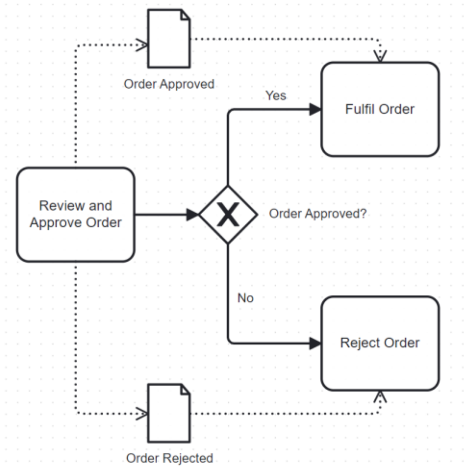
Message Flow



A Message Flow is used to show the flow of messages between two Participants of Process

- In BPMN, separate Pools are used to represent the Participants
- A Message Flow can connect to the boundary of the Pool or to an object within the Pool
- Message Flow are not allowed between objects within a single Pool

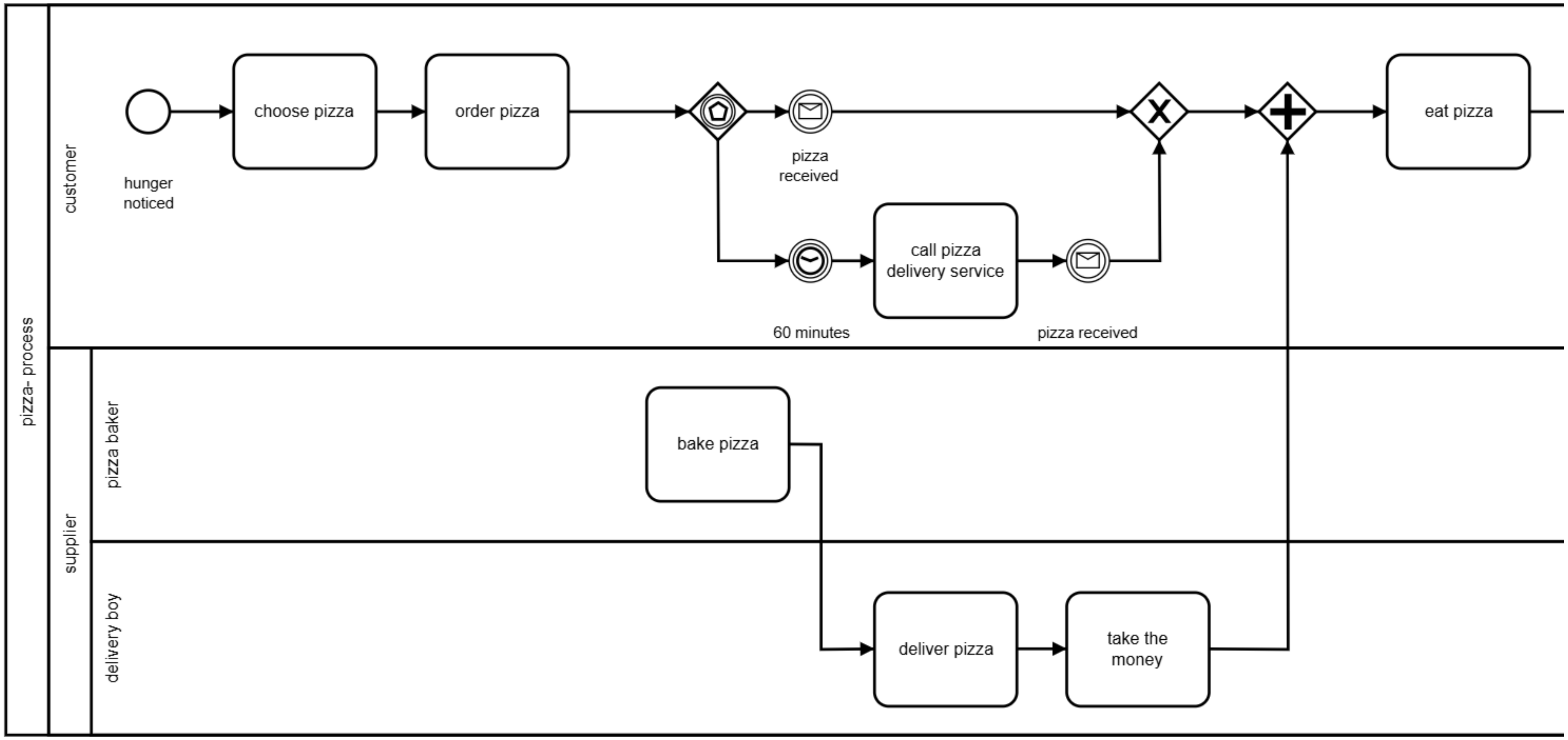
Association Flow



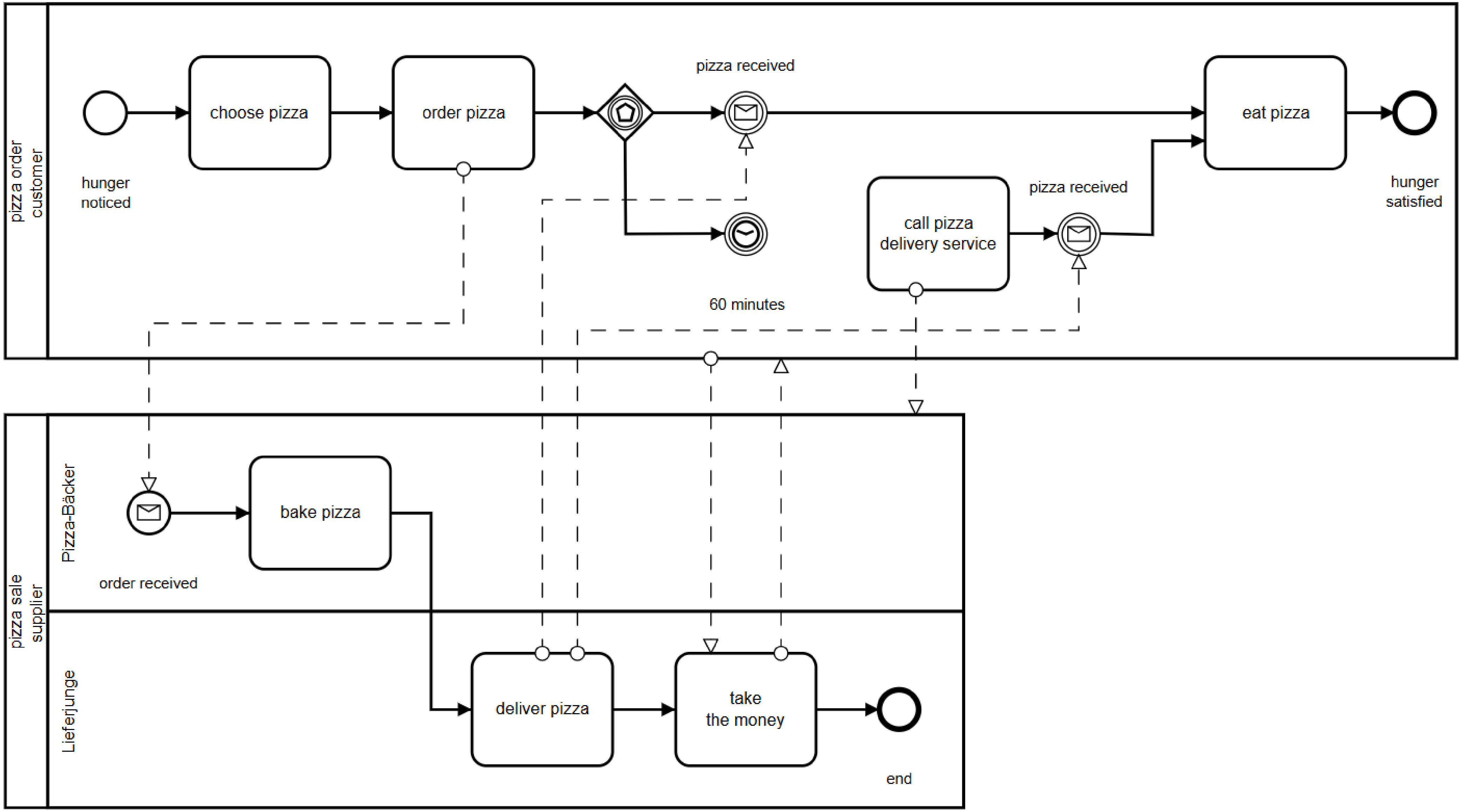
An Association is used to associate objects to one another (such as Artifacts and Activities)

Associations are used to show how data is input to and output from Activities
Text Annotations can be Associated with objects

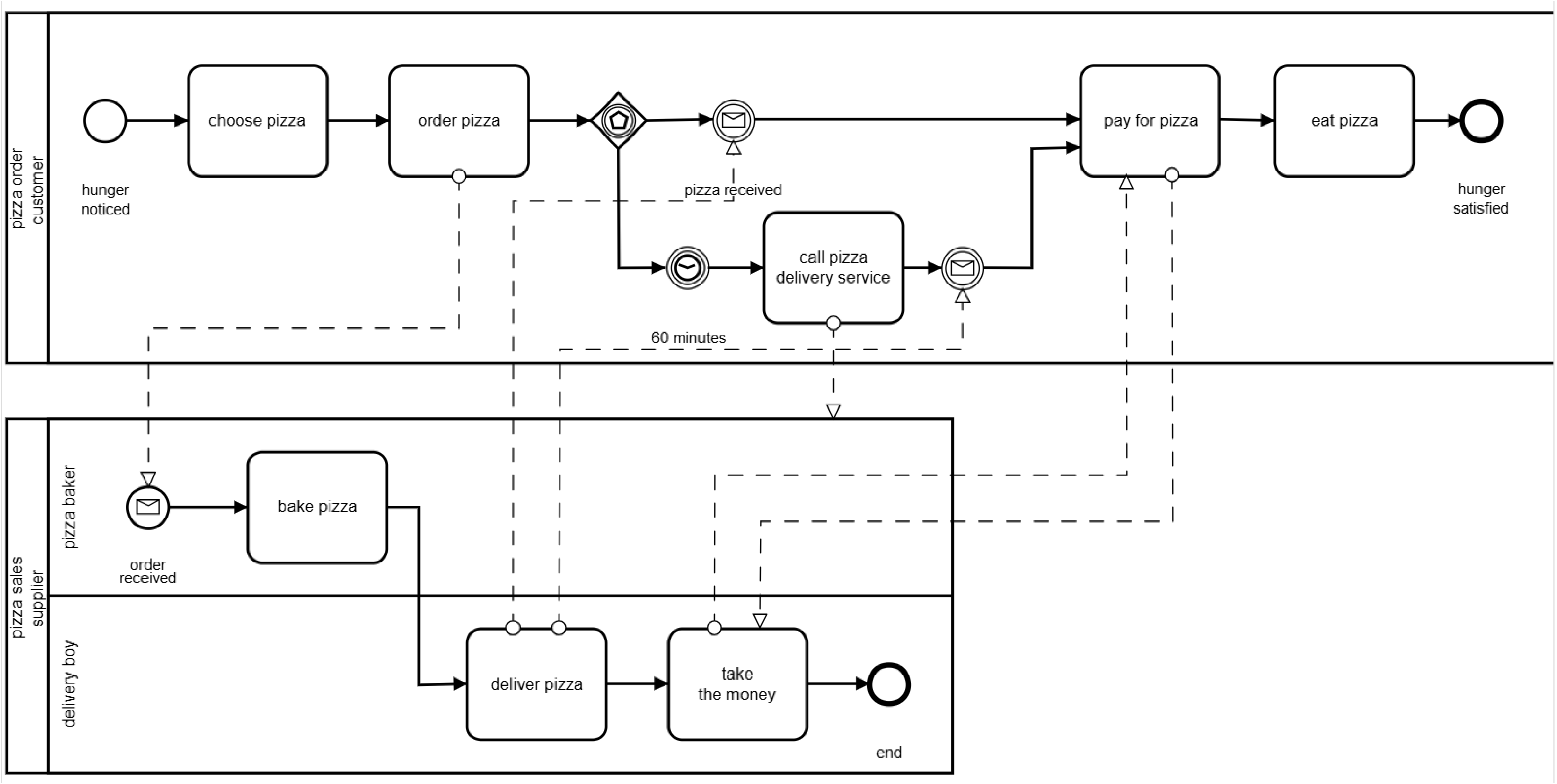
Connection Objects Example



Connection Objects Example



Connection Objects Example



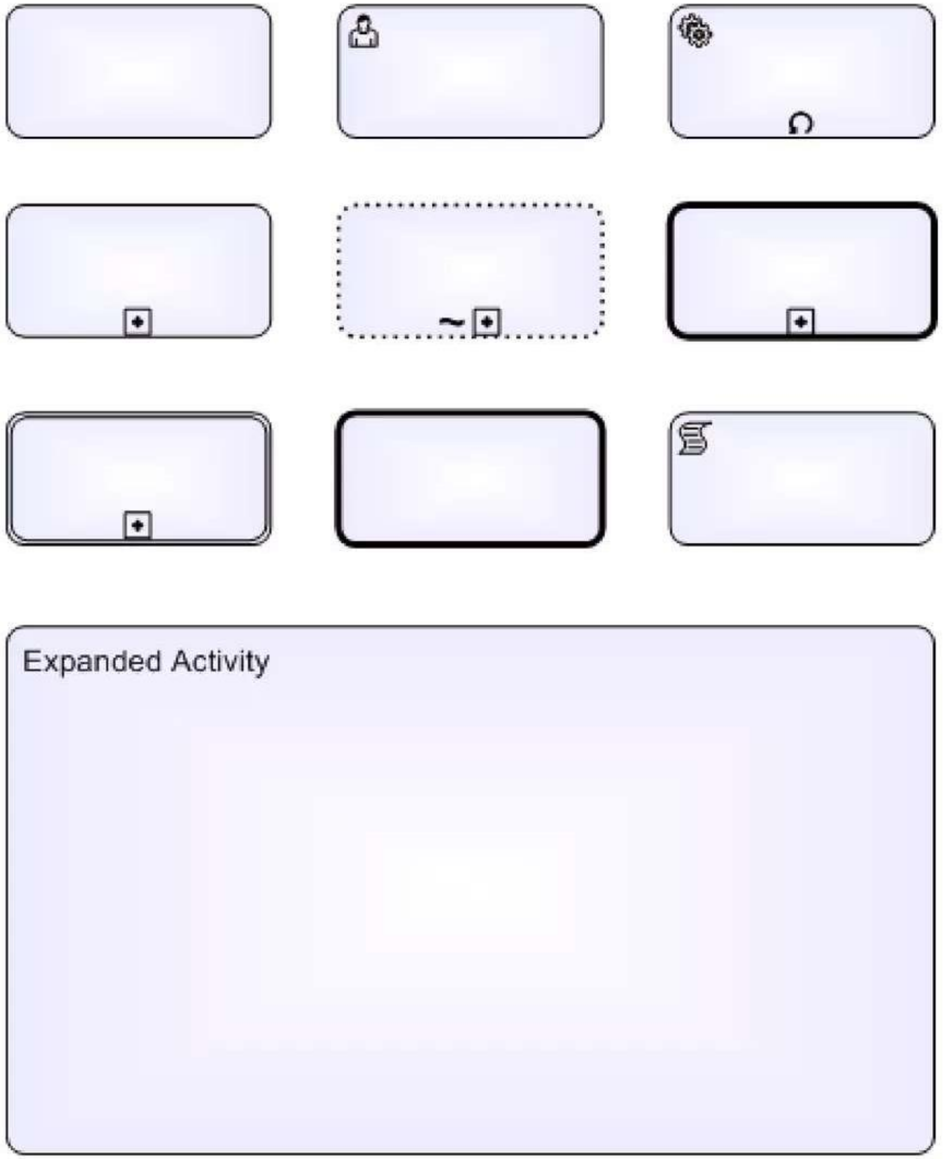
Activity

Activity is a generic term for work that company performs in a Process. An Activity Can be atomic or non-atomic.

The type of activities that are part of the process are: **Task** and **Sub-Process**.

A task can be differenced by markers that represent its type or associated resource.

Sub-Process can be Collapsed or Expanded, and can be differenced by the kind of elements that join in: **Sub-process**, **Transactions**, **Event Sub Process** and **Call Activities**.



Task Types

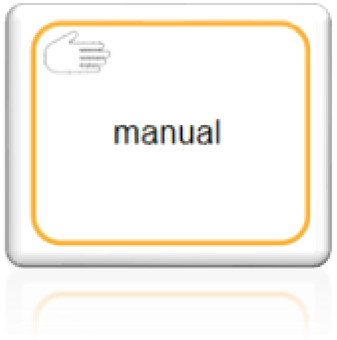
Receive Task

Receiving a message can be modeled as a separate task. This task type is an alternative to the catching message event, which is why the symbol for the event defined in BPMN 2.0 is an empty envelope.

Script Task

Scripts execute directly in the process engine, so they must be written in a language that the process engine can interpret.

Manual Task

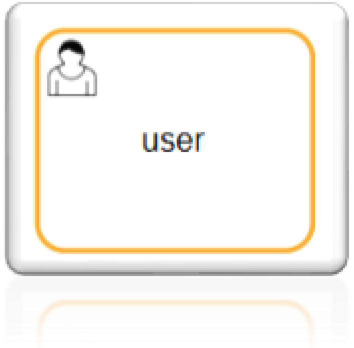


Tasks executed by a human being that do not affect the completion of a task assigned by the process engine.

Other examples: File a document in a folder, Clarify an incorrect invoice by phone, Talk with customers at the counter.

Task Types

User Task

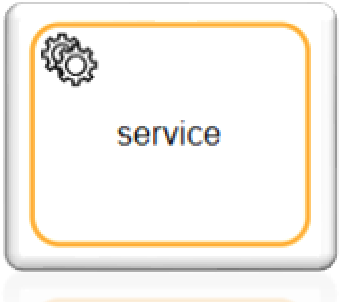


User tasks are executed by people just like the manual tasks, but they are assigned by a process engine, which may, for example, place these tasks in each user's task list.

After the human finishes a task, the engine expects confirmation, usually including data input or a button click. User tasks are part of the Human Workflow Management.

Typical task examples from the world of Human Workflow Management are: Check an invoice; Approve an application for vacation; Process a support request.

Service Task



Service tasks are those done by software. These are program functions applied automatically as a process executes. BPMN normally assumes that this function is provided as web service, though it can be another implementation. The service task is a component of process-oriented implementation integration, which explains why it is so similar in concept to Service-Oriented Architecture (SOA).

Typical examples from the world of implementation integration are: The credit rating provided by a rating agency, obtained as XML through HTTP during a credit check; Booking an invoice received as EDIFACT through X.400 in SAP R/3; The offer of substandard goods by an online auction house, as a web service.

Task Types

Send Task

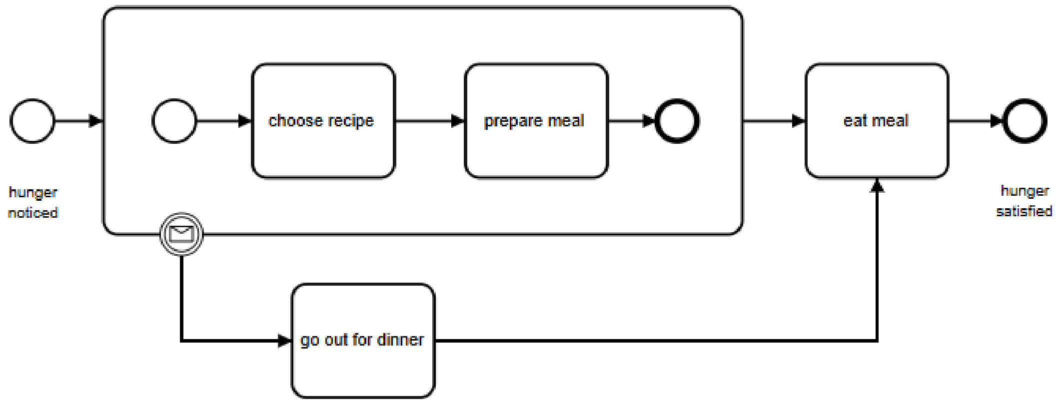
These tasks are technical, and the process engine executes them. Therefore, they mainly are used for calling web services asynchronously through message queues.

Business Rule Task

BPMN 2.0 provides another task type: the business rule. This task type is used solely to apply business rules.

Call Activity and Embedded Subprocess

Embedded Subprocess



An embedded subprocess can occur only within a parent process to which it belongs. An embedded subprocess cannot contain pools and lanes, but it can be placed within the pool or the lane of the parent process. Furthermore, an embedded subprocess may have only a none start event; start events such as messages or timers are not permitted.

An embedded subprocess has essentially nothing more than a kind of delimited scope within the parent process, which may serve two goals:

•To encapsulate complexity (as already described)

•To formulate a “collective statement” on a part of the parent process by attaching events

Call Activity

On the other hand, global subprocesses may occur in completely different parent processes.

The connection a global subprocesses has to its parent is considerably less close, and they can have their own pools and lanes.

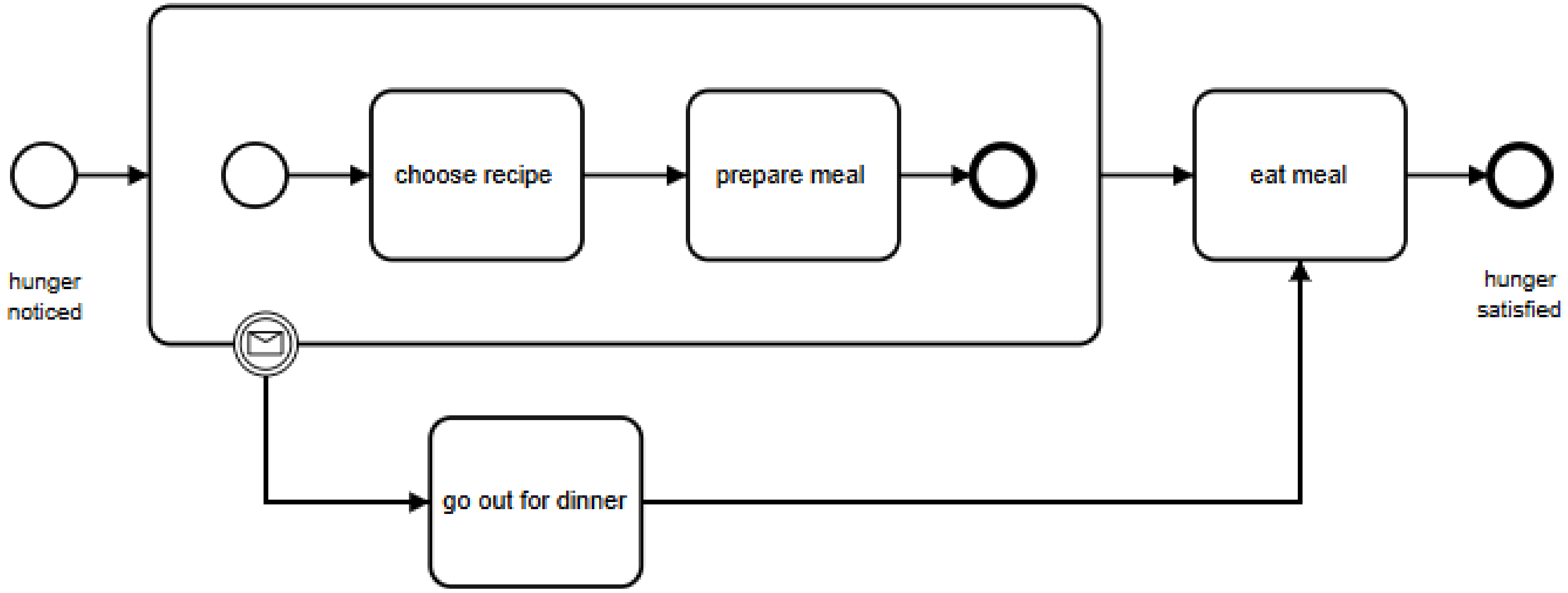
The loose connection also affects data transfer between the parent and the subprocess. BPMN assumes that embedded subprocesses can read all the data of the parent process directly, but an explicit assignment is required for global subprocesses to be able to read it.

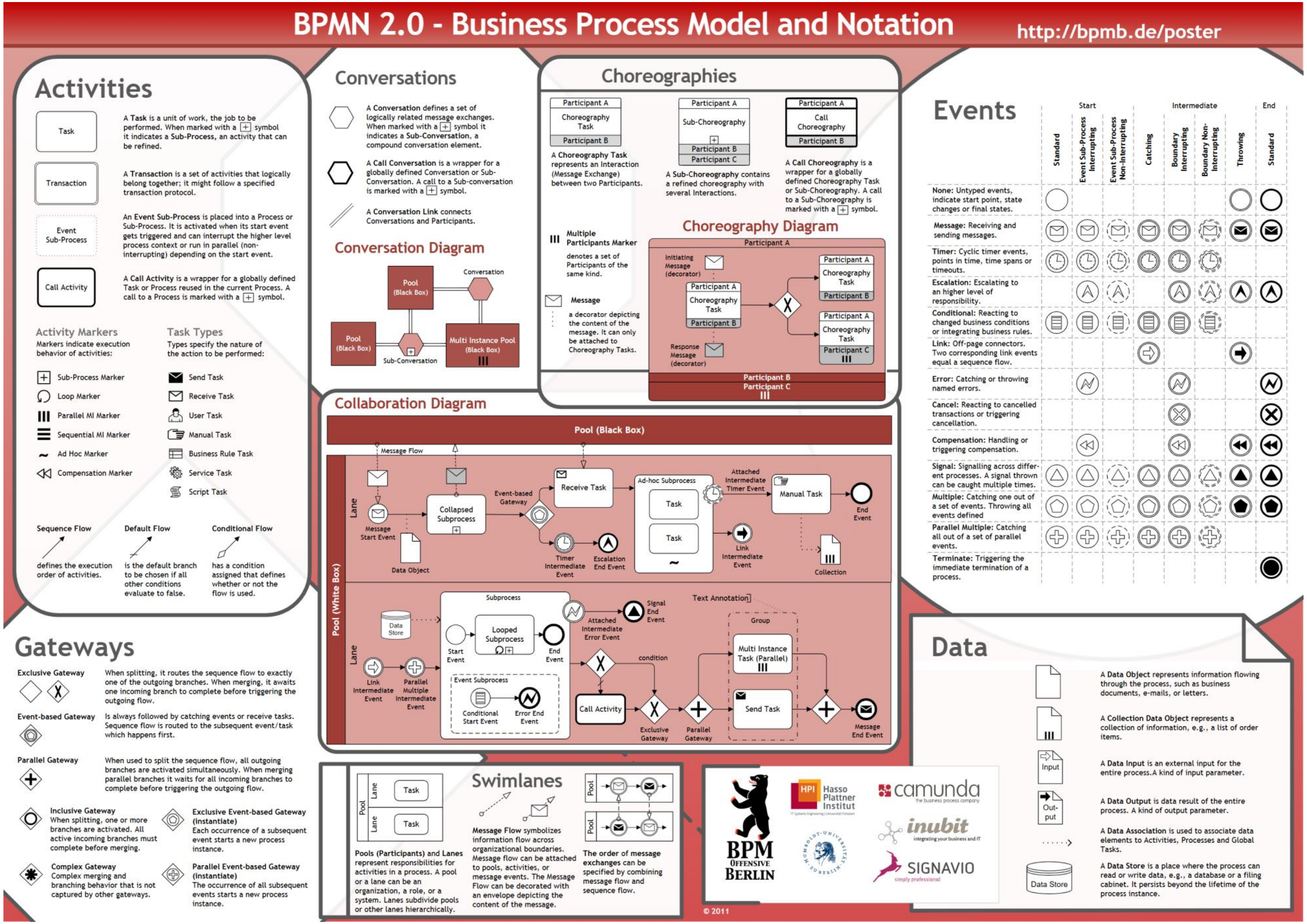
Call Activity and Embedded Subprocess

Embedded Subprocess

An embedded subprocess can occur only within a parent process to which it belongs. An embedded subprocess cannot contain pools and lanes, but it can be placed within the pool or the lane of the parent process. Furthermore, an embedded subprocess may have only a none start event; start events such as messages or timers are not permitted.

An embedded subprocess has essentially nothing more than a kind of delimited scope within the parent process, which may serve two goals:
•To encapsulate complexity (as already described)
•To formulate a “collective statement” on a part of the parent process by attaching events




BPMN 2.0 - Business Process Model and Notation
http://bpmb.de/poster
Activities
Task
A Task is a unit of work, the job to be performed. When marked with a ⊞ symbol it indicates a Sub-Process, an activity that can be refined.
Transaction
A Transaction is a set of activities that logically belong together; it might follow a specified transaction protocol.
Event Sub-Process
An Event Sub-Process is placed into a Process or Sub-Process. It is activated when its start event gets triggered and can interrupt the higher level process context or run in parallel (non-interrupting) depending on the start event.
Call Activity
A Call Activity is a wrapper for a globally defined Task or Process reused in the current Process. A call to a Process is marked with a ⊞ symbol.
Activity Markers
Markers indicate execution behavior of activities:
Sub-Process Marker
Loop Marker
Parallel MI Marker
Sequential MI Marker
Ad Hoc Marker
Compensation Marker
Task Types
Types specify the nature of the action to be performed:
Send Task
Receive Task
User Task
Manual Task
Business Rule Task
Service Task
Script Task
Sequence Flow
defines the execution order of activities.
Default Flow
is the default branch to be chosen if all other conditions evaluate to false.
Conditional Flow
has a condition assigned that defines whether or not the flow is used.
Gateways
Exclusive Gateway
When splitting, it routes the sequence flow to exactly one of the outgoing branches. When merging, it awaits one incoming branch to complete before triggering the outgoing flow.
Event-based Gateway
Is always followed by catching events or receive tasks. Sequence flow is routed to the subsequent event/task which happens first.
Parallel Gateway
When used to split the sequence flow, all outgoing branches are activated simultaneously. When merging parallel branches it waits for all incoming branches to complete before triggering the outgoing flow.
Inclusive Gateway
When splitting, one or more branches are activated. All active incoming branches must complete before merging.
Exclusive Event-based Gateway (instantiate)
Each occurrence of a subsequent event starts a new process instance.
Complex Gateway
Complex merging and branching behavior that is not captured by other gateways.
Parallel Event-based Gateway (instantiate)
The occurrence of all subsequent events starts a new process instance.
Conversations
A Conversation defines a set of logically related message exchanges. When marked with a ⊞ symbol it indicates a Sub-Conversation, a compound conversation element.
A Call Conversation is a wrapper for a globally defined Conversation or Sub-Conversation. A call to a Sub-conversation is marked with a ⊞ symbol.
A Conversation Link connects Conversations and Participants.
Conversation Diagram
Pool (Black Box)
Conversation
Pool (Black Box)
Sub-Conversation
Multi Instance Pool (Black Box)
Choreographies
Participant A
Choreography Task
Participant B
A Choreography Task represents an Interaction (Message Exchange) between two Participants.
Participant A
Sub-Choreography
Participant B
Participant C
A Sub-Choreography contains a refined choreography with several Interactions.
Participant A
Call Choreography
Participant B
A Call Choreography is a wrapper for a globally defined Choreography Task or Sub-Choreography. A call to a Sub-Choreography is marked with a ⊞ symbol.
Multiple Participants Marker
denotes a set of Participants of the same kind.
Message
a decorator depicting the content of the message. It can only be attached to Choreography Tasks.
Choreography Diagram
Participant A
Initiating Message (decorator)
Participant A
Choreography Task
Participant B
Participant A
Choreography Task
Participant B
Participant A
Choreography Task
Participant C
Response Message (decorator)
Participant B
Participant C
Collaboration Diagram
Pool (Black Box)
Message Flow
Pool (White Box)
Lane
Message Start Event
Collapsed Subprocess
Event-based Gateway
Receive Task
Ad-hoc Subprocess
Task
Task
Attached Intermediate Timer Event
Manual Task
End Event
Data Object
Timer Intermediate Event
Escalation End Event
Link Intermediate Event
Collection
Lane
Data Store
Subprocess
Start Event
Looped Subprocess
End Event
Attached Intermediate Error Event
Signal End Event
Text Annotation
Group
Link Intermediate Event
Parallel Multiple Intermediate Event
Event Subprocess
Conditional Start Event
Error End Event
condition
Multi Instance Task (Parallel)
Call Activity
Exclusive Gateway
Parallel Gateway
Send Task
Message End Event
Swimlanes
Pool
Lane
Task
Lane
Task
Pools (Participants) and Lanes represent responsibilities for activities in a process. A pool or a lane can be an organization, a role, or a system. Lanes subdivide pools or other lanes hierarchically.
Message Flow symbolizes information flow across organizational boundaries. Message flow can be attached to pools, activities, or message events. The Message Flow can be decorated with an envelope depicting the content of the message.
Pool
Pool
The order of message exchanges can be specified by combining message flow and sequence flow.
Events
Start
Intermediate
End
Standard
Event Sub-Process Interrupting
Event Sub-Process Non-Interrupting
Catching
Boundary Interrupting
Boundary Non-Interrupting
Throwing
Standard
None: Untyped events, indicate start point, state changes or final states.
Message: Receiving and sending messages.
Timer: Cyclic timer events, points in time, time spans or timeouts.
Escalation: Escalating to an higher level of responsibility.
Conditional: Reacting to changed business conditions or integrating business rules.
Link: Off-page connectors. Two corresponding link events equal a sequence flow.
Error: Catching or throwing named errors.
Cancel: Reacting to cancelled transactions or triggering cancellation.
Compensation: Handling or triggering compensation.
Signal: Signalling across different processes. A signal thrown can be caught multiple times.
Multiple: Catching one out of a set of events. Throwing all events defined
Parallel Multiple: Catching all out of a set of parallel events.
Terminate: Triggering the immediate termination of a process.
Data
A Data Object represents information flowing through the process, such as business documents, e-mails, or letters.
A Collection Data Object represents a collection of information, e.g., a list of order items.
Input
A Data Input is an external input for the entire process. A kind of input parameter.
Output
A Data Output is data result of the entire process. A kind of output parameter.
A Data Association is used to associate data elements to Activities, Processes and Global Tasks.
Data Store
A Data Store is a place where the process can read or write data, e.g., a database or a filing cabinet. It persists beyond the lifetime of the process instance.
BPM Offensive Berlin
Hasso Plattner Institut
camunda
inubit
SIGNAVIO
© 2011

References

References

- https://www.bmc.com/blogs/business-process-management-vs-automation/
- https://www.mega.com/blog/what-is-business-process-management-bpm
- https://appian.com/learn/topics/business-process-management/what-is-business-process-management
- https://camunda.com/glossary/business-process-management-bpm/
- https://en.wikipedia.org/wiki/List_of_Six_Sigma_companies
- https://boardmix.com/examples/example-of-lean-management/
- https://www.quora.com/What-is-the-difference-between-lean-six-sigma-and-lean-six-sigma#:~:text=The%20primary%20goal%20of%20Lean,the%20problem%20from%20different%20perspectives.
- https://www.smartsheet.com/all-about-six-sigma?srsltid=AfmBOorOx39qX5R0GxKgmaBG66XiUtHgw9yVFQC3ZzskIaRiR661at0V
- https://www.investopedia.com/terms/l/lean-six-sigma.asp
- https://goleansixsigma.com/reducing-lead-time-customer-replacement-part-orders-41/
- https://www.slideshare.net/slideshow/bpmn-20-introduction/15065684#2
- https://camunda.com/bpmn/reference
- https://www.slideshare.net/slideshow/bpmn-20-introduction/15065684#10
- https://www.slideshare.net/slideshow/omg-bpmn-tutorial-49508070/49508070
- https://therestaurantboss.com/bill-gates-automation-technology/#:~:text=%E2%80%9CThe%20first%20rule%20of%20any,the%20inefficiency.%E2%80%9D%20%E2%80%93%20Bill%20Gates
- https://www.geeksforgeeks.org/fishbone-diagram-example-use/